

Hochschule Esslingen

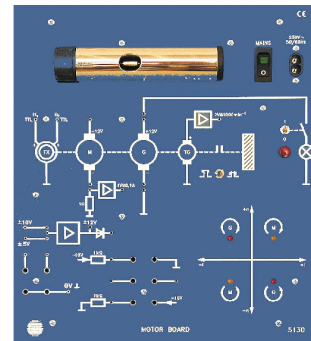
Fakultät Informationstechnik

Labor Regelungstechnik 2

Versuch: Motorboard

Bitte verwenden Sie

Matlab 2021a



Der Laborbericht ist spätestens eine Woche nach dem Labortermin abzugeben. Mangelhafte Vorbereitung beim Labortermin, verspätete oder unvollständige Abgabe des Laborberichts führen zum Nichtbestehen des Labors. Das Labor muss dann im folgenden Semester wiederholt werden.

Mit der Abgabe dieses Laborberichts und Ihrer Unterschrift bestätigen Sie, dass Sie den Bericht und die zugrundeliegenden Berechnungen, Programme und Simulationen selbstständig ohne Hilfe Dritter angefertigt haben.

Laborbericht von: _____

Laborpartner: _____

Motorboard Nr.: _____ Labortermin: _____

Die Laborteilnehmer sollten jeweils ihre **eigene handschriftlich** ausgearbeitete Laborvorbereitung in die Laborveranstaltung mitbringen.

Beachten Sie im Labor die aushängenden Sicherheitsvorschriften und die Sicherheitsunterweisung.

Inhaltsverzeichnis

1	Inhalt, Ziel und Ablauf des Versuchs	4
2	Versuchsaufbau	6
3	Beschreibung der verwendeten Komponenten	8
3.1	Motorboard HPS 5130	8
3.1.1	Gleichstrommotor M und Generator G	10
3.1.2	Strom-, Drehzahl- und Positionsmessung	10
3.1.3	Vierquadrant-Anzeige	11
3.2	Gleichstromquelle als Störgröße	12
3.3	Dragon12 Mikrocontrollerboard und Signalanpassschaltungen	13
3.3.1	Struktur des Hauptprogramms	13
3.3.2	PWM-Modul und Motoransteuerung	14
3.3.3	RTI Abtast-Interrupt und ADC-Messung von Strom- und Tachospannung	15
3.4	Anschluß des Motorboards an den Mikrocontroller	16
4	Modellierung und Parameteridentifikation	17
4.1	Simulationsmodell für die Strecke	17
4.2	Parameteridentifikation	21
4.2.1	Elektrischer Teil des Motors: R, L	21
4.2.2	Reibung	23
4.2.3	Trägheitsmoment	23
4.2.4	Motorkonstante k_M	25
4.3	Validierung des Simulationsmodells, Drehzahlmessgenauigkeit	26
5	Auslegung des Drehzahlreglers in der Simulation	29
5.1	Dimensionierung des PI-Drehzahlreglers	29
5.2	Verifikation des Drehzahlreglers in der Simulation	30
6	Implementierung des PI-Reglers auf dem Mikrocontroller	31
6.1	Umwandlung in einen zeitdiskreten PI-Algorithmus	31
6.2	Test des zeitdiskreten PI-Algorithmus in C im Simulink-Modell	31
6.3	Portieren des zeitdiskreten PI-Algorithmus auf das Dragon12-Board	32

7	Positionsregler	33
7.1	Dimensionierung und Test des Positionsreglers mit Matlab/Simulink	33
7.2	Implementierung der Positionsregelung auf dem Dragon12-Board	34
8	Anhang: Analyse von Messdaten	35
8.1	Software-Oszilloskop	35
8.2	Bedienung des Digital-Speicher-Oszilloskops	36
9	Anhang: Bedienoberfläche des HCS12-Programms	37
10	Anhang: Anschlüsse Motorboard - Mikrocontroller	38

Hinweis:

Die Simulink-Modelle für diesen Laborversuch und die hier abgebildeten Diagramme wurden mit Matlab R2021a erstellt. Neuere Versionen sind in der Regel aufwärtskompatibel, können sich aber in Details unterscheiden. Bei Problemen verwenden Sie bitte die Version R2021a.

1 Inhalt, Ziel und Ablauf des Versuchs

Der Laborversuch Motorboard findet aufgeteilt in zwei Laborterminen statt.

Thema: Entwurf einer Drehzahl- sowie einer Lageregelung in Kaskadenstruktur für einen Gleichstrommotor. Der Regler wird dabei mit Hilfe eines HCS12-Mikrocontrollers auf einem Dragon12 Entwicklungsboard realisiert.

Inhalt:

- Analyse der vorgegebenen Systemkomponenten
- Modellbildung und Parameteridentifikation
- Validierung des Simulationsmodells
- Reglerentwurf und Inbetriebnahme auf dem Mikrocontrollerboard
- Realisierung der Regler mit automatischer Codegenerierung und in Integerarithmetik

Ziel: In diesem Laborversuch werden die folgenden Inhalte der Vorlesung Regelungstechnik vertieft und praktisch angewendet:

- Modellbildung
- Simulation, Parameteridentifikation und Validierung des Simulationsmodells
- Reglerentwurf und -realisierung

Ablauf des Laborversuchs

Die Laborversuche bestehen jeweils aus einem Vorbereitungsteil und einem Teil, der erst im Labor durchgeführt wird. Die Vorbereitungsteile enthalten eine umfangreiche Anzahl von Aufgaben, die vor dem Laborversuch bearbeitet werden müssen und in den Umdrucken mit **VORBEREITUNG** gekennzeichnet sind. Bei der Vorbereitung sind sowohl Aufgaben und Berechnungen manuell zu lösen als auch Simulationen mit Matlab/Simulink durchzuführen.

1. **Notwendige Vorkenntnisse und Vorbereitungsaufwand:** Für die Vorbereitung dieses Laborversuchs muss ausreichend Zeit eingeplant werden. Insbesondere benötigen Sie für dieses Labor solide Kenntnisse im Umgang mit Matlab/Simulink aus vorhergehenden Laborversuchen der Vorlesungsreihe Regelungstechnik sowie Kenntnisse zu HCS12 und Codewarrior aus der Vorlesung Computerarchitektur. Bringen Sie zum Laborversuch unbedingt die entsprechenden Vorlesungsunterlagen mit.
2. **Bearbeitung des Laborumdrucks:** [Den Laborumdruck sowie vorbereitete Matlab/Simulink-Dateien finden Sie im Moodle-Kurs zur Vorlesung.](#) Tragen Sie im Umdruck an den dafür vorgesehenen Stellen Ihre Lösungen handschriftlich in die freien Felder ein und ergänzen Sie bei Bedarf durch Zusatzblätter. Bringen Sie zum Laborversuch sämtliche Vorbereitungsunterlagen inklusive der Simulationsdateien mit. Berechnungen sollen handschriftlich, die Simulationsergebnisse müssen ausgedruckt und beschriftet vorliegen!

Die Vorbereitung sollte in der Laborgruppe gemeinsam so durchgeführt werden, dass jeder Gruppenteilnehmer in der Lage ist, die Lösung zu sämtlichen Vorbereitungsaufgaben selbständig zu erklären und bei Bedarf sämtliche Simulationsprogramme vorzuführen und die Ergebnisse zu erläutern.

Mangelhafte Vorbereitung oder Nicht-Teilnahme führt zum Nicht-Bestehen des Labors. Das Labor kann in diesem Fall im folgenden Semester wiederholt werden.

3. **Projektverzeichnis im Labor:** Legen Sie bitte zu Beginn des Labors auf der Festplatte des Laborrechners ein temporäres Verzeichnis an, in das Sie alle Ihre Daten aus Ihrer Laborvorbereitung kopieren. Der Zugriff auf lokale Verzeichnisse ist üblicherweise deutlich schneller als auf Netzlaufwerke.
4. **Laborbericht:** Jede Studentengruppe gibt spätestens eine Woche nach dem Versuch einen vollständigen Bericht zum Labor ab. Dies kann bei sorgfältiger Vorbereitung schon direkt nach dem Labortermine erfolgen. Eventuell müssen jedoch vor der Abgabe einzelne Punkte noch nachgearbeitet werden.

Der Laborbericht umfasst diesen Laborumdruck mit den handschriftlich (!) eingetragenen Lösungen sowie die Zusatzblätter mit den beschrifteten und kommentierten Simulationsergebnissen sowie gegebenenfalls Nebenrechnungen. Bei allen Unterlagen muss eindeutig erkennbar sein, zu welcher Aufgabe sie gehören, welche Signale (bei Simulationsergebnissen) dargestellt sind bzw. welche Parameter verwendet wurden. Diagrammachsen müssen beschriftet und skaliert werden. Bei Zahlenangaben sind die Einheiten mit anzugeben.

Heften Sie die Unterlagen in einen Schnellhefter oder ein Ringbuch ein. Loseblattsammlungen werden als Bericht nicht anerkannt!

Beachten Sie bitte, dass die Vorbereitung und der Laborbericht nicht eine stupide Aneinanderreihung von Simulationsausdrucken sein sollten. Die Berechnungen und vor allem die Simulationen sind nur dann sinnvoll, wenn Sie die Aufgabenstellung und den Lösungsweg verstanden haben und die Simulationsergebnisse interpretieren und kommentieren. Nur so können Sie das gewünschte Lernziel erreichen.

Bitte bringen Sie zum Labortermine auch die Unterlagen aus der Vorlesung *Computerarchitektur* mit, die Sie zur Bedienung der Codewarrior-Entwicklungsumgebung und zur Programmierung des HCS12 mit den verwendeten Peripheriekomponenten benötigen.

Sicherheitsvorschriften

Bitte beachten Sie die aushängenden Sicherheitsvorschriften im Labor und folgen Sie den Anweisungen der Laborbetreuer. Bei Fragen und Problemen wenden Sie sich an die zuständigen Laboringenieure, Herrn Peter oder Herrn Zins, bzw. an Prof. Lindermeir oder Prof. Zimmermann.

2 Versuchsaufbau

In diesem Kapitel soll ein Überblick über den Versuchsaufbau, die verwendeten Komponenten und ihre Verschaltung gegeben werden. Das Ersatzschaltbild der Strecke ist in Abbildung 1 dargestellt.

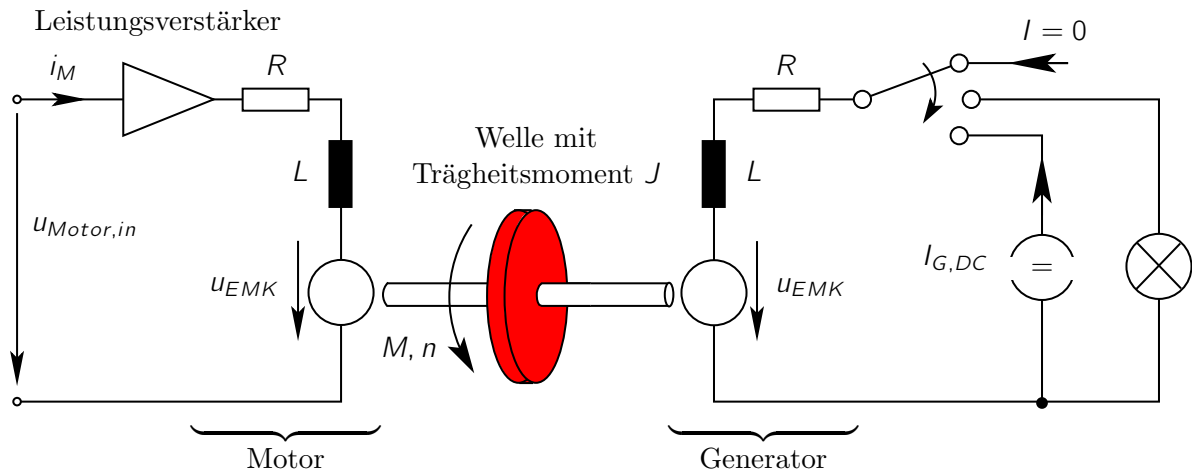


Abbildung 1: Ersatzschaltbild der Regelstrecke (Motorboard)

Im Bild links ist ein **Leistungsspannungsverstärker** und ein permanenterregter **Gleichstrommotor** abgebildet, der starr an eine Welle angekoppelt ist. Im rechten Teil ist ein Generator zu sehen, der baugleich mit dem Motor ist und ebenfalls starr an die Welle gekoppelt ist. Für den Motor gelten prinzipiell folgende Zusammenhänge:

- Das Drehmoment ist proportional zum Motorstrom, d.h. $M \sim i_M$
- Die induzierte Spannung ist proportional zur Drehzahl, d.h. $u_{EMK} \sim n$
Über die angelegte Spannung $u_{Motor,in}$ kann man die Motordrehzahl steuern. Um die Drehrichtung umzukehren ($n < 0$), muss man eine negative Eingangsspannung $u_{Motor,in}$ anlegen.
- Wenn $u_{EMK} \sim n$ und $i_M \sim M$ dasselbe Vorzeichen haben, spricht man von **Motorbetrieb**, d.h. der Motor nimmt elektrische Leistung auf und gibt mechanische Leistung ab.
Wenn $u_{EMK} \sim n$ und $i_M \sim M$ unterschiedliche Vorzeichen haben, spricht man von **Generatorbetrieb**, d.h. der Motor nimmt mechanische Leistung auf und gibt elektrische ab.

Der **Generator** erzeugt ein Belastungsmoment für den Antriebsmotor. Der Strom durch den Generator und damit das Belastungsmoment kann auf drei Arten beeinflusst werden:

- Falls der Schalter oben ist, kann kein Generatorstrom fließen. Dadurch kann der Generator kein Drehmoment auf die Welle ausüben.
- Falls der Schalter in der mittleren Stellung ist, speist der Generator eine Glühlampe (= konstanter Widerstand). Dadurch erzeugt der Generator immer ein der momentanen Bewegungsrichtung entgegengesetztes Lastmoment an der Welle, d.h. er wirkt bremsend.

- Falls der Schalter unten ist, wird durch eine geregelte Stromquelle ein vorgegebener Strom $I_{G,DC}$ in den Generator eingeprägt. Hierdurch kann ein definiertes Laststörmoment eingeprägt werden, das je nach Richtung des eingeprägten Stromes positiv oder negativ sein kann.

Zur Regelung verwenden wir den bekannten HCS12 Mikrocontroller auf dem Dragon12 Board. Ein Übersichtsbild des gesamten Versuchsaufbaus ist in Abbildung 2 dargestellt.

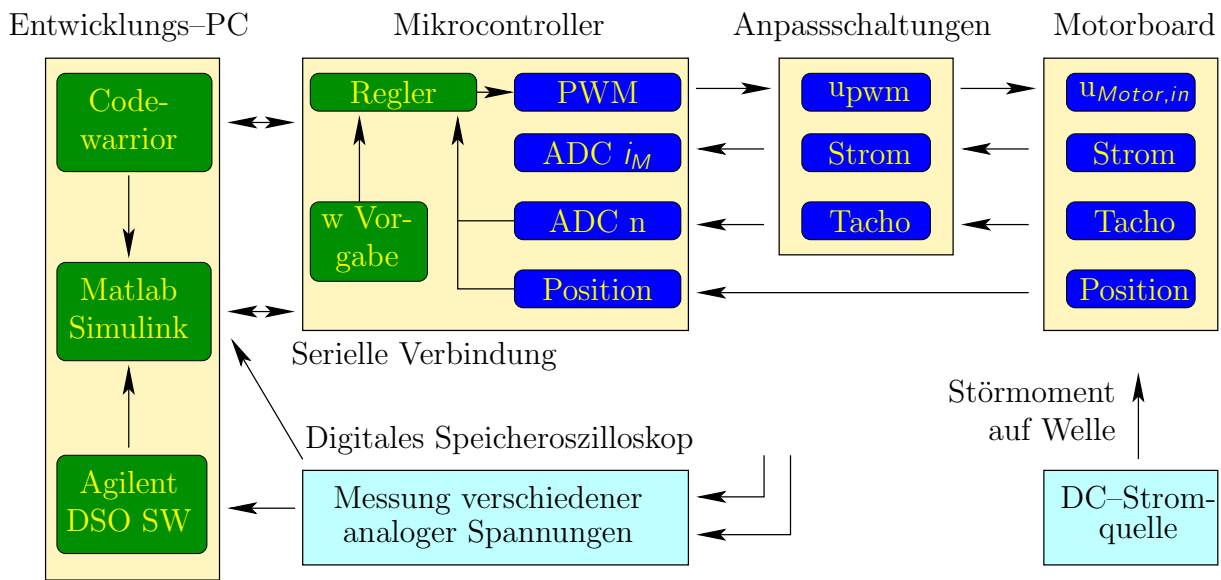


Abbildung 2: Blockschaltbild des gesamten Versuchsaufbaus.

Der Mikrocontroller steuert über seinen PWM-Ausgang die Eingangsspannung am Motorboard. Die Messgrößen sind der Strom i_M durch den Motor, die Drehzahl n und die Position φ der Welle. Diese Größen werden teilweise über Anpassschaltungen (Spannungsteiler, Verstärker) geführt. Im Mikrocontroller wird ein Regelalgorithmus in Kaskadenstruktur entworfen, der basierend auf den Messsignalen Drehzahl und Position die Ansteuerung der Motoreingangsspannung bestimmt. Der Motorstrom wird in dieser Anordnung nur gemessen, um eine Überlast des Motors zu erkennen und falls erforderlich abzuschalten.

Als zusätzliche Hardware-Komponenten werden eine Gleichstromquelle zur Realisierung einer Laststörung und ein digitales Speicheroszilloskop (DSO) von Agilent zur Messung von analogen Spannungen verwendet.

Im Mikrocontroller ist eine Software-Komponente eingebunden, die eine serielle Verbindung zu Matlab auf dem Entwicklungs-PC herstellt. Über diese Verbindung können maximal zwei Signale in Echtzeit an Matlab weitergegeben werden, um dort beispielsweise graphisch dargestellt zu werden. Dies ist die einzige Möglichkeit, die Position der Welle in der Entwicklungsumgebung am PC verfügbar zu machen. In Anhang 8.1 ist beschrieben, wie das entsprechende Vorgehen hierzu ist.

Die Modellierung, Simulation, Reglerauslegung und der Vergleich zwischen Messung und Simulation findet am Entwicklungs-PC mit Matlab/Simulink statt.

Vorbereitungsaufgabe 1: Simulationsmodell

Zum besseren Verständnis der Anwendung können Sie das Simulink-Modell *Regelkreis_ZK.slx* öffnen.

- Bevor Sie die Simulation starten, führen Sie bitte zunächst das M-File *params_m.m* aus, um die Parameterwerte zu initialisieren, und stellen Sie die Schalter im Simulink-Modell so ein, dass das gewünschte Modell entsteht (Positionsregler, Drehzahlregler oder Handbetrieb). Beginnen Sie mit der Drehzahlreglung.
- Mit einem weiteren Schalter (im Modell des Motorboards) können Sie die Störgröße ein- oder abschalten. Die Lampe in Abbildung 1 ist hier nicht modelliert.
- Rechts oben befinden sich zwei Animationsblöcke. Diese können zur besseren Simulationsperformance ausgeschaltet werden. Klicken Sie die Blöcke an und schalten Sie sie ein (Häkchen bei *Enable-CheckBox*) oder aus.
- Der Block Tacho stellt einen Drehzahlmesser dar, an dem Sie die aktuelle Drehzahl in *U/min* sehen können.
- Der Block Linearachse stellt eine Achse dar, die die rotatorische Bewegung der Welle in eine translatorische Bewegung umsetzt. An der Achse kann die Positionierung der Welle gut erkannt werden. Darüber hinaus kann über die Buttons eine Sollposition für den Positionsregler vorgegeben werden.

Wenn Sie die Simulationen ausführen, sollten die Regler schon halbwegs sinnvolles Verhalten zeigen. Der Positionsregler schwingt deutlich über. In diesem Laborversuch werden wir dies ändern. Zu dieser Vorbereitungsaufgabe müssen Sie nichts abgeben.

3 Beschreibung der verwendeten Komponenten

Da in diesem Laborversuch ein reales Regelungssystem betrachtet wird, werden zunächst die verwendeten Systemkomponenten untersucht. Im Anschluß daran findet die Modellbildung statt, die Voraussetzung für den Regelungsentwurf ist.

3.1 Motorboard HPS 5130

Das Motorboard HPS 5130 ist in Abbildung 3 dargestellt. Oben sehen Sie die Welle mit dem Motor und dem Generator. Die elektrischen Anschlüsse dieser Komponenten sind bei den jeweiligen Symbolen durch Stecker zugänglich. Den 3-fach Schalter aus Abbildung 1 realisieren wir wie folgt:

- Schalter oben: Steckbrücke SB ziehen oder Schalter SL auf 0.
- Schalter mitte: Steckbrücke SB einstecken und Schalter SL auf 1.

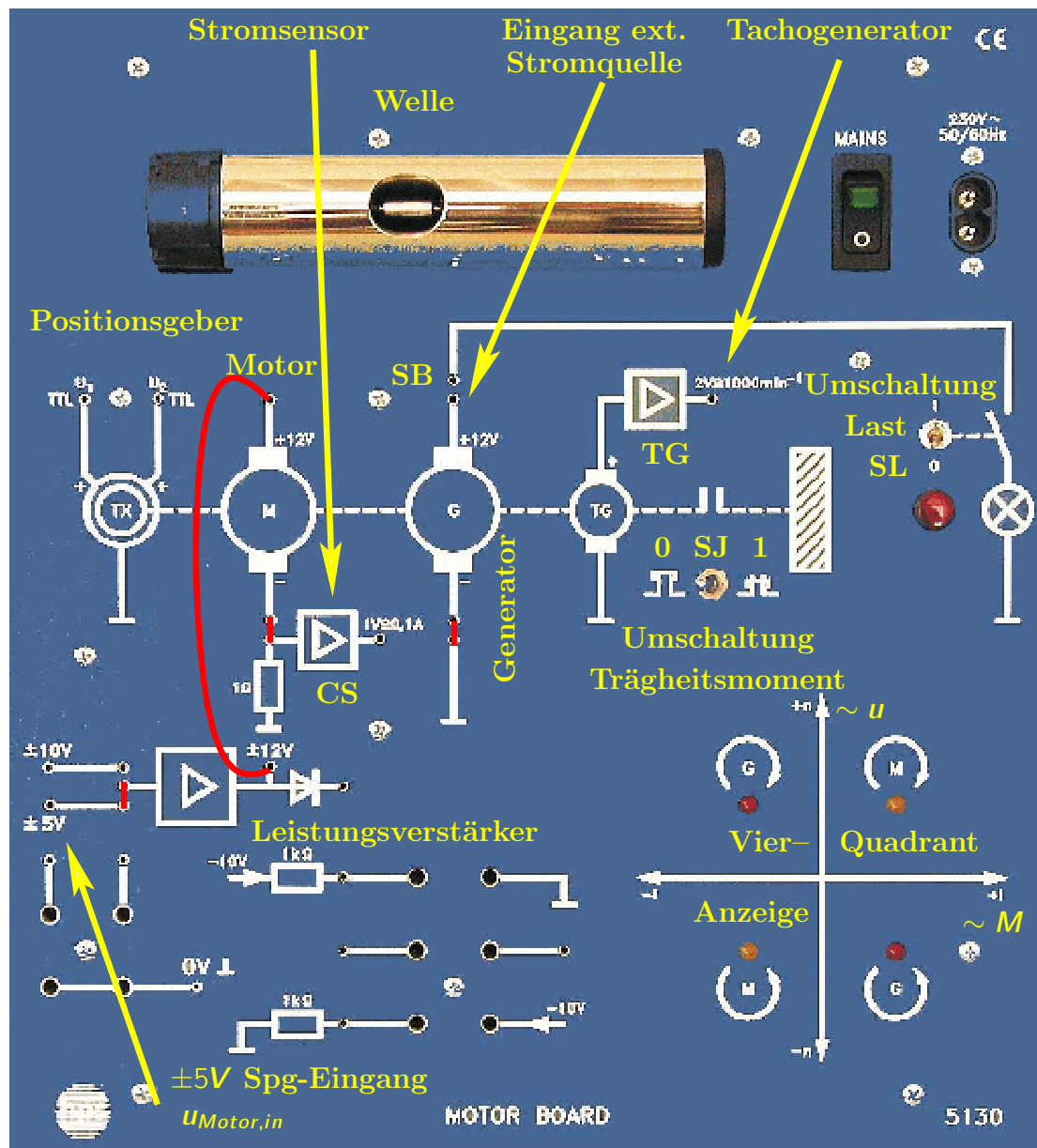


Abbildung 3: Motorboard 5130 der Firma HPS als Regelstrecke. Die offenen Buchsen sind im Labor bei Bedarf mit kleinen Steckbrücken oder Kabeln kurzgeschlossen (teilweise in rot angedeutet).

- Schalter unten: Steckbrücke SB ziehen und externe Stromquelle beim unteren Kontakt einspeisen.

Das Trägheitsmoment der Welle kann über den Schalter SJ verändert werden. In Position 0 hat das Trägheitsmoment der Welle den Wert $J = J_{K(lein)}$. In Position 1 wird das Trägheitsmoment auf den Wert $J = J_{G(roß)}$ vergrößert (wird auf dem Motorboard elektronisch nachgebildet).

Wicklungswiderstand	R	$11,9\Omega$
Induktivität	L	keine Angabe
Motorkonstante	k_m	$14,8 \cdot 10^{-3} \frac{Nm}{A}$
Nennspannung	U_N	$12V$
Leerlaufstrom bei $u = U_N$	I_0	$20mA$
Leerlaufdrehzahl bei $u = U_N$	n_0	$7800 \frac{U}{min}$
Max. Dauerstrom	I_{max}	$580mA$

Tabelle 1: Parameter des Motors bzw. des Generators

3.1.1 Gleichstrommotor M und Generator G

Der Motor und der Generator sind permanenterregte Gleichstrommotoren vom gleichen Typ. Die elektrischen Daten des Motors bzw. Generators aus dem Datenblatt des Herstellers finden Sie in Tabelle 1. Im Verlauf des Labors werden diese Größen teilweise nachgemessen.

3.1.2 Strom-, Drehzahl- und Positionsmessung

Stromsensor CS: Der Motorstrom i_M wird durch die Messung des Spannungsabfalls an einem Messwiderstand $R_{shunt} = 1\Omega$ (Englisch: Shunt) ermittelt und das Messsignal mit einem Messverstärker mit $1V$ pro $100mA$ verstärkt.

Tachogenerator TG: Die Motordrehzahl n wird über einen Tachogenerator in eine proportionale Spannung gewandelt. Die Spannung am Verstärkerausgang beträgt $2V$ pro $1000 \frac{U}{min}$.

Positionsgeber TX: Der Drehwinkel φ wird digital erfasst. Wenn sich die Welle dreht, erzeugt der Positionsgeber zwei TTL-Rechtecksignale U_1 und U_2 (Abbildung 4), deren Phasen um 90° gegeneinander versetzt sind (Inkremental- oder Quadratur-Positionsgeber). Jedes Signal hat 100 Impulse je Umdrehung, insgesamt also 400 Signalfanken je Umdrehung. Die Messung ist inkrementell (relativ), eine absolute Nullposition existiert bei diesem Geber nicht.

Wir verwenden diesen Positionsgeber in zwei unterschiedlichen Anwendungen im Labor:

1. Falls U_1 bzw. U_2 mit einem Digitalspeicheroszilloskop gemessen und die Welle bei konstanter Drehzahl betrieben wird, kann aus der Periodendauer T_{TTL} der gemessenen Spannung auf die Drehzahl der Welle geschlossen werden (eine Umdrehung pro $100 \cdot T_{TTL}$).
2. Zur Positionsbestimmung der Welle wird im Mikrocontroller ein Treiber implementiert, der die positiven und negativen Flanken der beiden Signale U_1 und U_2 erkennt und eine Zählvariable jeweils inkrementiert (Vorwärtsfahrt) bzw. dekrementiert (Rückwärtsfahrt) und so den (relativen) Drehwinkel der Welle bestimmt. Die Drehrichtung wird dabei aus dem Pegel des jeweils anderen Signals ermittelt. Die Auflösung der Positionsmessung beträgt

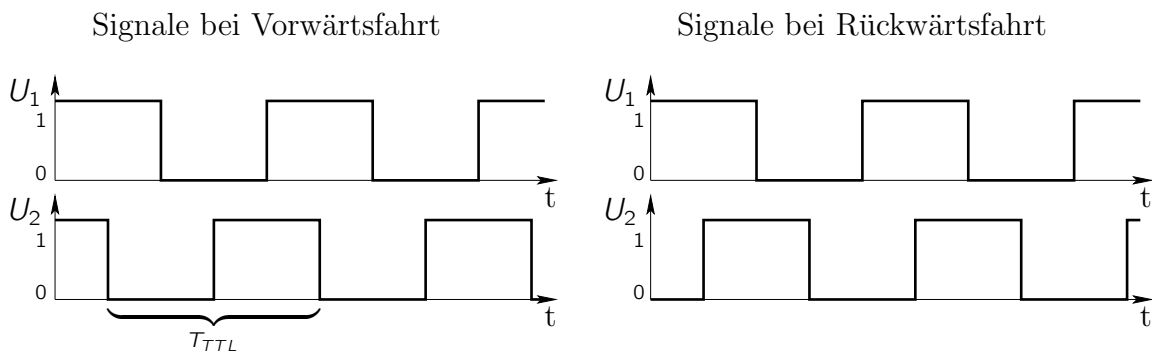


Abbildung 4: Quadratur-Ausgangssignale des Positionsgebers bei const. Drehzahl.

$360^\circ/400 = 0.9^\circ$. Das Problem, dass man lediglich relative Winkeländerungen erfassen kann, wird in der Applikation dadurch umgangen, dass man einen definierten Punkt anfährt (z.B. einen mechanischen Anschlag) und dort den inkrementellen Positionszähler initialisiert.

3.1.3 Vierquadrant-Anzeige

Unten rechts am HPS Motorboard befindet sich eine sogenannte Vierquadranten-Anzeige mit LEDs, die die Drehrichtung und die Richtung des Drehmoments anzeigt:

Drehmoment $M \sim i_M$	Drehzahl $n \sim u_{EMK}$	Betriebsart
> 0 (positiv)	> 0 (positive Drehrichtung)	Motorbetrieb (1.Quadrant)
< 0 (negativ)	> 0 (positive Drehrichtung)	Generatorbetrieb (2.Quadrant)
< 0 (negativ)	< 0 (negative Drehrichtung)	Motorbetrieb (3.Quadrant)
> 0 (positiv)	< 0 (negative Drehrichtung)	Generatorbetrieb (4.Quadrant)

Im Motorbetrieb wird elektrische Leistung $P_{elektr.} = u_{EMK} \cdot i_M$ in mechanische Leistung $P_{mech} = M \cdot 2\pi n$ umgesetzt, im Generatorbetrieb mechanische Leistung in elektrische Leistung (vergl. Abbildung 1). In beiden Fällen geht ein (hier relativ großer) Teil der Leistung als elektrische Verlustleistung $P_{V,el} = R \cdot i_M^2$ und als mechanische Verlustleistung durch (drehzahlabhängige) Reibung verloren.

Vorbereitungsaufgabe 2: Berechnung der stationären Drehzahl des Motors

Berechnen Sie aus den Daten in Tabelle 1 und dem Ersatzschaltbild in Abbildung 1 die an den Motorklemmen aufgenommene elektrische Nennleistung $P_{el,N}$ bei Nennspannung und maximalem Dauerstrom. (Achtung: Bei allen Zahlenwerten Einheiten nicht vergessen!)

$$P_{el,N} =$$

Wie groß wird bei Nennleistung die elektrische Verlustleistung $P_{V,el,N}$ am Wicklungswiderstand?

$$P_{V,el,N} =$$

Im Leerlaufbetrieb müsste der Motorstrom eigentlich 0 sein. In Wirklichkeit fließen laut Datenblatt aber $I_0 = 20mA$. Rechnen Sie für den Leerlauf ebenfalls die an den Motorklemmen aufgenommene elektrische Leistung $P_{el,0}$ sowie die Verlustleistung $P_{V,el,0}$ am Wicklungswiderstand aus.

$$P_{el,0} =$$

$$P_{V,el,0} =$$

Die Differenz dieser beiden Leistungswerte entspricht ungefähr den mechanischen Reibungsverlusten $P_{mech,0}$. Man darf davon ausgehen, dass die mechanischen Verluste bei einem Elektromotor im Wesentlichen nur von der Drehzahl, aber nicht vom Drehmoment abhängen, d.h. das Reibungsmoment $M_R = d \cdot 2\pi n$ ist näherungsweise proportional zur Drehzahl (sogenannte viskose Reibung). Den Proportionalitätsfaktor, den sogenannten Reibbeiwert $d = \frac{P_{mech,0}}{(2\pi n_0)^2}$ kann man dann aus den Reibungsverlusten berechnen.

Die induzierte Spannung des Motors im Leerlauf bei Nennspannung lässt sich aus der Maschengleichung abschätzen: $U_{EMK,0} = U_N - R \cdot I_0 = 12V - 11,9\Omega \cdot 20mA = 11,76V$. Die Drehzahl n_0 im Leerlauf beträgt laut Datenblatt 7800U/min. Bei Nennspannung und max. Dauerstrom sinkt die induzierte Spannung auf $U_{EMK,N} = 12V - 11,9\Omega \cdot 580mA \approx 5,1V$ ab. Da die induzierte Spannung der Drehzahl proportional ist, kann man aus dem Spannungsverhältnis die Nenndrehzahl des Motors n_N ausrechnen, die im Datenblatt leider nicht angegeben wurde. Wie groß ist sie?

$$n_N =$$

Wegen des relativ großen Wicklungswiderstandes sinkt gerade bei kleinen Motoren die Drehzahl bei Belastung gegenüber dem Leerlauf sehr stark ab. Um dies zu verhindern, ist eine Drehzahlregelung notwendig, die dem Drehzahleinbruch durch eine passende Vergrößerung der Klemmenspannung entgegenwirkt.

3.2 Gleichstromquelle als Störgröße

Die verwendete Stromquelle kann maximal einen Strom von $\pm 200mA$ bei einer Spannung von bis zu 20V einprägen. Da der maximal erlaubte Dauerstrom unseres Motors $I_{max} = 580mA$ beträgt, kann der Motor durch den Strom der Stromquelle eigentlich nicht beschädigt werden. Allerdings beträgt die Nennspannung des Motors nur 12V, während die Stromquelle bis zu 20V liefern kann. Um den Motor zu schützen, der bei höheren Spannungen ohne Belastung eine Drehzahl deutlich über seiner Nenndrehzahl annehmen könnte, wird die Stromquelle im Labor daher auf eine Maximalspannung von 12V eingestellt.

3.3 Dragon12 Mikrocontrollerboard und Signalanpassschaltungen

Wie in der Vorlesung *Computerarchitektur* verwenden wir auch hier den HCS12 Mikrocontroller der Firma Freescale auf einem Dragon12 Entwicklungsboard mit der Codewarrior-Entwicklungsumgebung. Nachfolgend sind die verwendeten Peripherie-Komponenten beschrieben, die wir als Interface-Schaltungen zum Motorboard benötigen. Bitte greifen Sie bei Bedarf auf die in den genannten Vorlesungen verwendeten Dokumentationen zurück, da die Module hier nur sehr knapp beschrieben werden.

Es werden drei verschiedene Codewarrior-Projekte verwendet:

- *Lab1_uC/Systemtechnik.mcp*: Regler in Gleitkomma-Arithmetik (wird in diesem Laborversuch verwendet).
- *Lab2_uC.autocode/Systemtechnik.mcp*: Regler in Gleitkomma-Arithmetik und automatischer Code-Generierung (wird erst im zweiten Laborversuch benötigt).
- *Lab2_uC.integer/Systemtechnik.mcp*: Regler in Integer-Arithmetik (wird erst im zweiten Laborversuch benötigt).

3.3.1 Struktur des Hauptprogramms

Um den generellen Aufbau des vorgegebenen Hauptprogramms besser verstehen zu können, sollen Sie das Projekt *./uC/Systemtechnik.mcp* zunächst analysieren.

Vorbereitungsaufgabe 3: Analyse der Struktur des Hauptprogramms

In den Dateien *main.c* und *utils.c* finden Sie zwei Zustandsautomaten mit den Aufzählungstypen *control_state* und *display_state*, die den Betriebsmodus der Regelung (Drehzahlregelung, Lageregelung usw.) festlegen. Die Bedienung des Programms erfolgt über die Taster am Port H des Mikrocontrollers.

Ermitteln Sie zunächst für jeden der beiden Automaten die Namen der möglichen Zustände und in welcher Routine die Zustände verändert werden.

Geben Sie das sich ergebende Zustandsdiagramm für den *control_state*-Automaten in Abbildung 5 und für den *display_state*-Automaten in Abbildung 6 an. Sie werden bei der Analyse sehen, dass das Verhalten der Automaten u.a. von einem Fehlerzustand abhängt. Die Abhängigkeit vom Fehlerzustand können Sie in den Zustandsdiagrammen weglassen. Wenn Ihnen beim *display_state*-Automaten eine Besonderheit auffällt, können Sie dies in Abbildung 6 dokumentieren.

Wie kann ein Fehlerzustand quittiert werden:

Hinweis: Sehen Sie sich zur Bearbeitung dieser Vorbereitungsaufgabe auch die Beschreibung in Anhang 9 an.

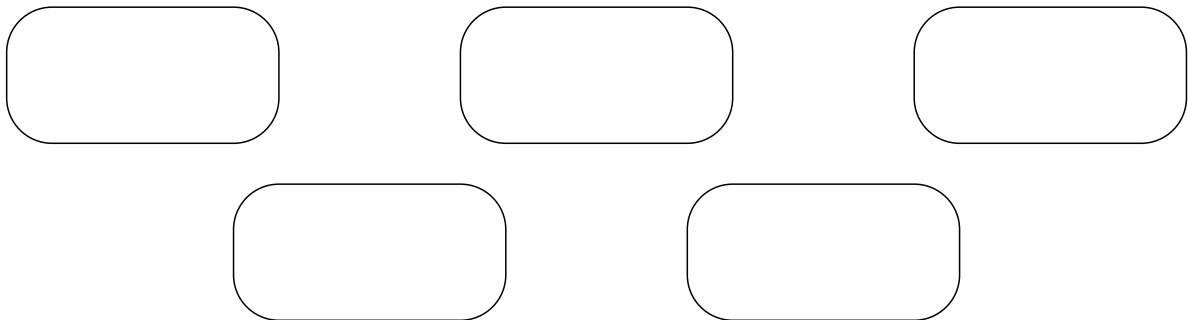


Zustandsübergänge jeweils

Zustandsübergänge in

implementiert

Abbildung 5: Zustandsdiagramm des *control_state*-Automaten



Zustandsübergänge jeweils

Zustandsübergänge in

implementiert

Sonderfall bei Zustand *set_val_0_min* bei Positionsregelung:

Abbildung 6: Zustandsdiagramm des *display_state*-Automaten

3.3.2 PWM-Modul und Motoransteuerung

Um den Motor mit einer analogen Spannung $u_{Motor,in}$ ansteuern zu können, verwenden wir das PWM-Modul des Mikrocontrollers. Unser HCS12 besitzt 8 PWM-Ausgabekanäle. Im Labor wird Kanal 0 mit einer Periodendauer des PWM-Signals von $42\mu sec$ verwendet. In Abhängigkeit des 8-bit Wertes des Tastverhältnisses kann der Gleichanteil der Ausgangsspannung dabei von 0V (PWM-Wert = 0) bis 5V (PWM-Wert = 255) in 255 Schritten verändert werden.

Da der Eingang des Motorboards aber eine Spannung $u_{Motor,in}$ im Wertebereich $\pm 5V$ erwartet, ist

noch eine Anpassschaltung (Signal-Conditioning) nötig, weil der Motor sonst nur in eine Richtung drehen könnte. Das PWM-Signal wird daher mit 2 verstärkt und durch eine Offset-Spannung von 5V in den geforderten Bereich verschoben (Abbildung 7). Außerdem wurde in die Anpassschaltung ein Tiefpassfilter für das PWM-Rechtecksignal integriert, um die vom PWM-Signal erzeugten EMV-Störungen zu reduzieren.

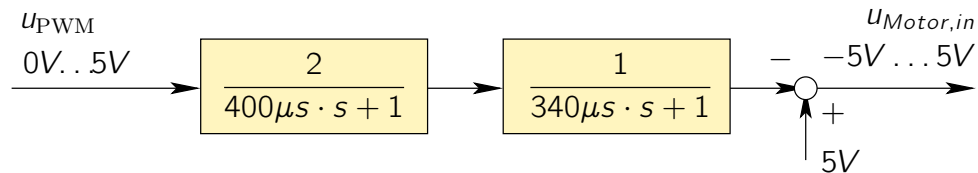


Abbildung 7: Blockschaltbild der PWM-Ansteuerung des Motors

Vorbereitungsaufgabe 4: Motorspannung bei gegebener PWM-Ansteuerung

Tragen Sie in Tabelle 2 den Zusammenhang zwischen dem PWM-Wert (8bit unsigned), dem Mittelwert der PWM-Ausgangsspannung $\overline{U_{PWM}}$ und der Motorspannung $u_{Motor,in}$ ein.

PWM-Wert	$\overline{U_{PWM}}$	$u_{Motor,in}$
0		
128		
255		

Tabelle 2: PWM-Rechenwert und Motorspannung (stationär)

3.3.3 RTI Abtast-Interrupt und ADC-Messung von Strom- und Tachospannung

Das RTI Modul (Real-Time-Interrupt) wird verwendet, um periodisch nach der Abtastzeit $T \approx 1536\mu s$ einen Interrupt auszulösen. In der Interrupt Service Routine erfolgt der Start der AD-Wandlungen für die Spannung des Tachogenerators und des Stromsensors (vergl. Kapitel 3.1.2). Auch diese Signale werden durch Anpassschaltungen gefiltert und an den Spannungsbereich (0...5V) des Analog-Digital-Umsetzers angepasst (Abbildung 8 und 9). Verwendet werden die ADC-Kanäle 0 und 1, das Wandlungsergebnis ist rechtsbündig mit 10-bit Auflösung. Für jedes Signal wird der Median aus drei Messungen gebildet, um Messausreißer zu unterdrücken.

Nach erfolgter Wandlung generiert das ADC-Peripheriemodul einen Interrupt. Die zugehörige ADC-Interrupt-Service-Routine (ISR) verarbeitet nicht nur die Messwerte, sondern enthält auch die Regelalgorithmen für die Drehzahl bzw. Position des Motors.

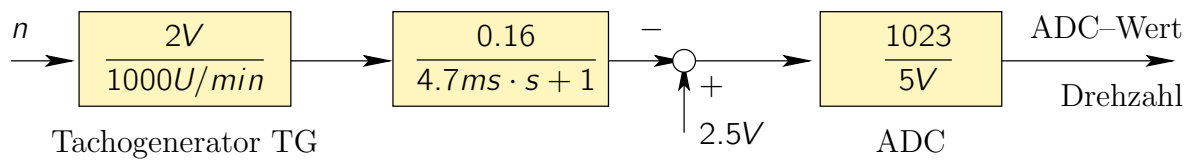


Abbildung 8: Blockschaltbild der Signal-Anpassschaltung für die Tachospannung

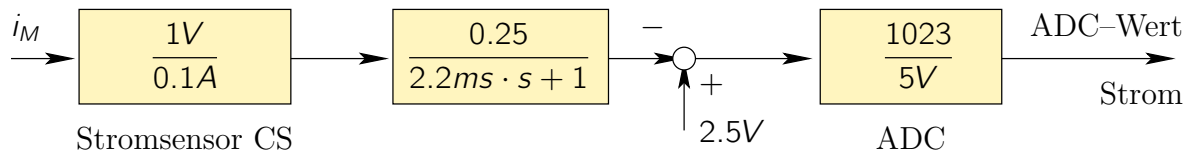


Abbildung 9: Blockschaltbild der Signal-Anpassschaltung für den Stromsensor

Vorbereitungsaufgabe 5: ADC-Werte bei gegebenen Drehzahlen und Strömen

Tragen Sie in Tabelle 3 den Zusammenhang zwischen Drehzahl bzw. Strom und den jeweiligen ADC-Messwerten ein.

Drehzahl n	ADC-Wert
	1023
$-7000 \frac{U}{min}$	
$0 \frac{U}{min}$	
$+7000 \frac{U}{min}$	
	0

Strom i_M	ADC-Wert
$-1A$	
$0A$	
$+1A$	

Tabelle 3: ADC-Rechenwerte bei gegebenen Drehzahlen und Strömen (stationär)

3.4 Anschluß des Motorboards an den Mikrocontroller

Siehe Anhang S. 38

widerstand R (siehe Tabelle 1) und dem Widerstand R_{shunt} des Stromsensors (siehe 3.1.2) zusammen.

$$R_m = \quad \quad \quad \text{und} \quad T_{el,m} =$$

Die oberen drei, bisher unvollständig angeschlossenen Blöcke im Blockschaltbild sollen den Generator modellieren, der das Last-Störmoment für den Motor erzeugt. Wie schon im Ersatzschaltbild in Abbildung 1 und im Simulink-Modell (Seite 8) beschrieben, kann der Generator entweder aus einer Konstantstromquelle $I_{G,DC}$ gespeist oder an eine Glühlampe als Lastwiderstand ($R_{Lampe} = 12,4\Omega$) angeschlossen werden.

- Wenn die Konstantstromquelle verwendet wird, wird zusätzlich zum Reibmoment M_r ein Belastungsmoment $M_L = k_m \cdot I_{G,DC}$ auf den Motor wirken.
- Falls dagegen die Glühlampe aktiv ist, wird das Belastungsmoment

$$M_L = k_m \frac{\frac{1}{R_g}}{sT_{el,g} + 1} \cdot \underbrace{k_m \cdot \omega}_{u_{G,EMK}: \text{induzierte Spg. am Generator}}$$

$$\text{mit } R_g = R + R_{Lampe} = \quad \quad \quad \text{und } T_{el,g} = \frac{L}{R_g} =$$

wirksam. Berechnen Sie die Zahlenwerte für R_g und $T_{el,g}$.

Tragen Sie in die entsprechenden Blöcke im Blockschaltbild nach Abbildung 10 die Übertragungsfunktionen und Verstärkungsfaktoren zur Beschreibung des Generators ein und zeichnen Sie die entsprechenden Verbindungsleitungen ein. Die Umschaltung zwischen den beiden Fällen für M_L soll durch einen Schalter erfolgen.

Im Simulink-Modell *Strecke.slx* fehlen die Blöcke zur Modellierung des Generators noch, die Sie soeben ergänzt haben. Bauen Sie die fehlenden Blöcke in das Modell ein und ergänzen Sie eventuell für das Modell benötigte Parameter in der Datei *strecke_params.m*, mit der dieses Modell vor der Simulation initialisiert werden muss. Führen Sie dann zur Verifikation Ihrer Änderungen am Modell Simulationen mit folgenden Werten durch:

- Simulationsdauer $t_e = 5\text{sec}$
- Sprung der Eingangsspannung zur Zeit $t_w = 1\text{sec}$
- Sprung der Stromquelle zur Zeit $t_v = 3\text{sec}$:

Folgende Fälle sollen simuliert werden:

- a. Sprung von $u_{Motor,in}$ von 0V auf 4V, kein Strom $I_{G,DC}$ durch den Generator.
- b. Sprung von $u_{Motor,in}$ von 0V auf 4V, Generator an Lampe.

- c. Sprung von $u_{Motor,in}$ von 0V auf 4V, Generator an Stromquelle mit Sprung von 0 auf $I_{G,DC} = +200mA$.
- d. Sprung von $u_{Motor,in}$ von 0V auf 4V, Generator an Stromquelle mit Sprung von 0 auf $I_{G,DC} = -200mA$.

Die Simulationsdiagramme sollen in getrennten Diagrammen jeweils den Motorstrom i_M und die Motordrehzahl n darstellen. Die Diagramme erstellen Sie am besten mit dem Matlab `plot()`-Befehl. Die zugehörigen Signale werden aus dem Simulink-Modell durch die To Workspace-Blöcke exportiert. Vergessen Sie die Beschriftung der Diagramme nicht!

Versuchen Sie die Simulationsergebnisse zu verstehen. Zum Vergleich sind in Abbildung 11 und 12 die Drehzahlverläufe für die Fälle b und c dargestellt.

Warum ist der Endwert des Stroms im Fall a nicht Null, obwohl es kein Störmoment gibt?

Antwort:

Warum ist die Enddrehzahl im Fall b deutlich kleiner als im Fall a?

Antwort:

Welche unterschiedliche Wirkung hat das Störmoment im Fall c und im Fall d?

Antwort:

Aus den Simulationsergebnissen sollten Sie eventuelle Fehler in der Modellierung erkennen und korrigieren können. Falls Ihr Simulationsergebnis zu Fall c) anders aussieht als in Abbildung 12, müssen Sie den Fehler unbedingt suchen und beseitigen, weil sonst nachfolgende Simulationen möglicherweise ebenfalls falsch werden!

Laboraufgabe 0: Überprüfen des Laboraufbaus

Schalten Sie das Motorboard ein, laden Sie das Codewarrior-Projekt im Originalzustand auf das Dragon12-Board und starten Sie es. Stellen Sie mit Hilfe der Bedientasten (sh. S.37) den Manual-Modus ein und aktivieren Sie den eingebauten Rechteckgenerator, um Spannungssprünge an den Motor anzulegen. Starten Sie in Matlab das *SoftScope* (sh. S.35), um den Drehzahlverlauf zu messen. Wahrscheinlich müssen Sie beim Starten des Debuggers und beim Starten des *SoftScopes* jeweils die korrekten seriellen Schnittstellen konfigurieren.

Wenn Sie die Drehzahländerungen hören (!) und sehen können, funktioniert der Aufbau korrekt und Sie können mit den eigentlichen Laboraufgaben beginnen.

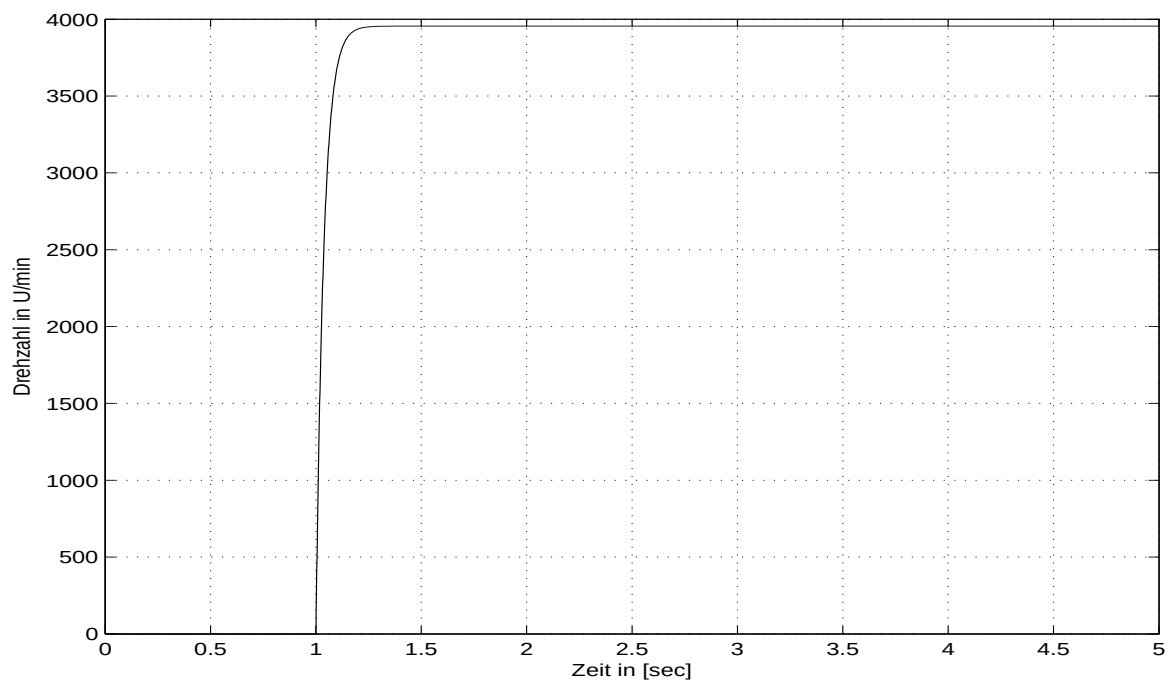


Abbildung 11: Drehzahlverlauf zu Vorbereitungsaufgabe 6.b

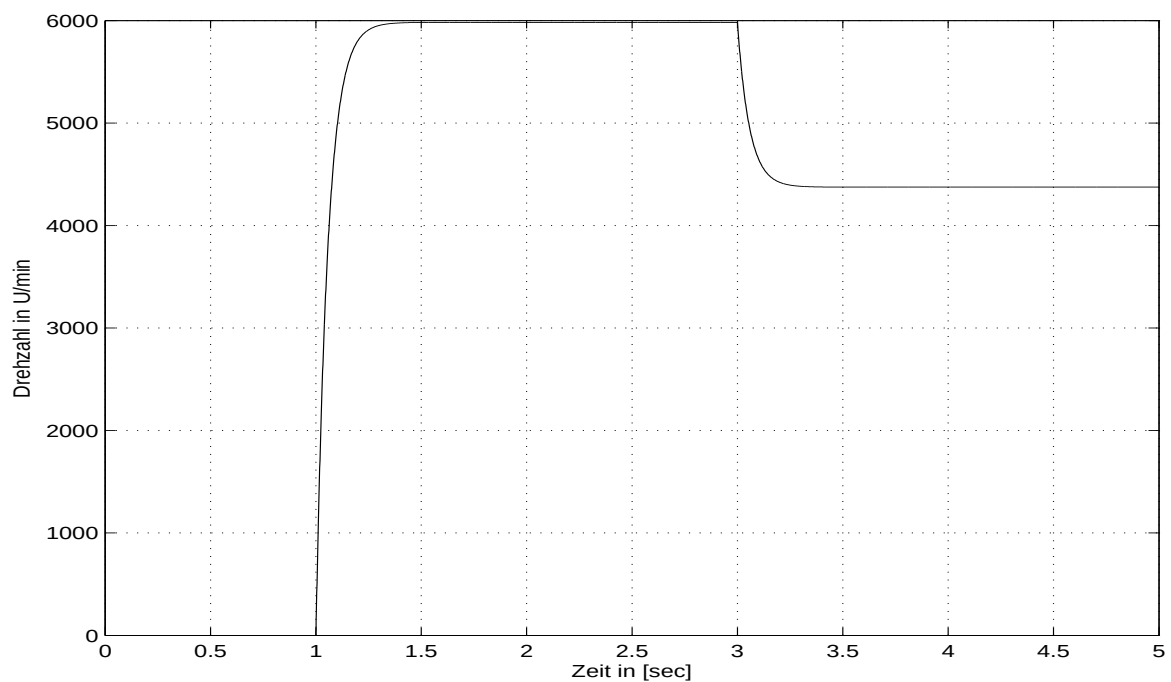


Abbildung 12: Drehzahlverlauf zu Vorbereitungsaufgabe 6.c

4.2 Parameteridentifikation

Unter **Systemidentifikation** versteht man das Finden einer zu den Messungen passenden Systemstruktur und deren Parametrierung. Man kann z.B. die Sprungantwort oder den Frequenzgang einer Strecke messen und dann eine Übertragungsfunktion suchen, die zu den Messwerten passt.

In der Praxis erhält man allerdings wesentlich bessere Ergebnisse, wenn man, wie im vorliegenden Fall, ein auf Gleichungen aufbauendes physikalisches Modell für die Strecke aufstellt. Dann muß man „nur“ diejenigen Parameter des physikalischen Modells, die nicht in Datenblättern zu finden sind, durch **Parameteridentifikation**, d.h. aus Messungen, gewinnen. Durch den Vergleich der Messergebnisse mit Simulationsmodellen kann und muß man in der Praxis auch überprüfen, ob das Simulationsmodell korrekt ist.

Der „Trick“ bei der Parameteridentifikation besteht darin, genau solche Messungen durchzuführen, die die Wirkung jedes Modellparameters möglichst unabhängig von den anderen Parametern zeigen. Im Folgenden werden wir diskutieren, mit welchen Messungen die Motorparameter identifiziert werden können. Einen Teil dieser Messungen werden Sie selbst im Labor durchführen.

4.2.1 Elektrischer Teil des Motors: R, L

Am einfachsten sind die elektrischen Daten des Motors zu bestimmen. Mit einem Ohmmeter können wir prüfen, ob der im Datenblatt angegebene Wert korrekt ist. Da es Exemplarstreuungen in der Fertigung gibt und der Wicklungswiderstand von der Temperatur abhängt, sind kleinere Abweichungen zu erwarten. An einem Exemplar im Labor wurde bei Raumtemperatur beispielsweise ein Widerstand von $R = 11.2\Omega$ gemessen (Datenblatt: $R = 11.9\Omega$).

Zur Bestimmung der Induktivität können wir die Sprungantwort des Motorstroms i_M bei einem Sprung der Motorspannung u_{PWM} messen. Im ersten Augenblick nach dem Sprung wird sich die Motordrehzahl und damit auch u_{EMK} noch nicht ändern, da die mechanische Zeitkonstante wesentlich größer sein dürfte als die elektrische. Damit gilt für die Sprungantwort der in Abbildung 13 dargestellte Ausschnitt aus dem gesamten Blockschaltbild des Motors. Da wir weiter annehmen wollen, dass die Zeitkonstanten der Anpassschaltung sehr viel kleiner sind, erwarten wir im Wesentlichen PT1-Verhalten mit der Zeitkonstante $T_{el,m}$.

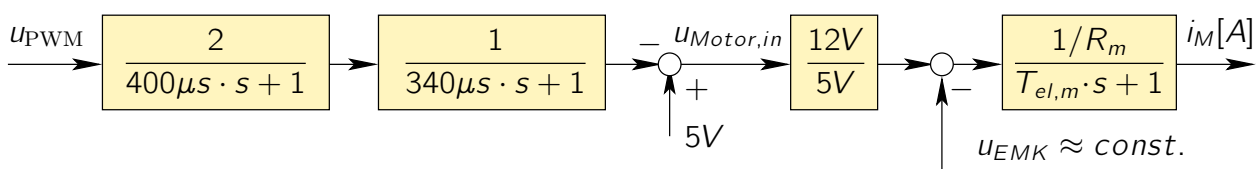
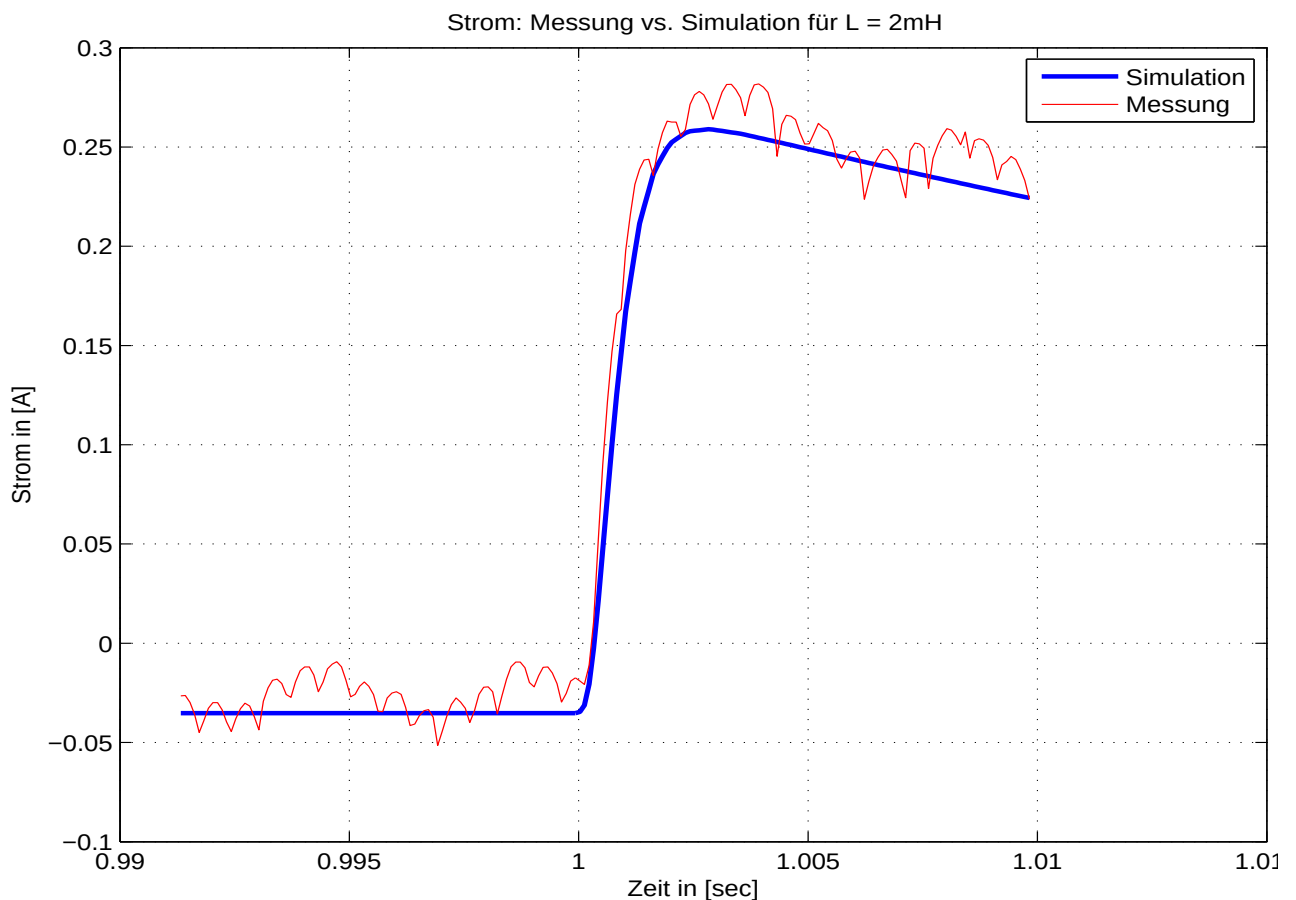


Abbildung 13: Blockschaltbild zur Identifikation der Motorinduktivität

Abbildung 14 zeigt das Ergebnis einer Messung und den Vergleich mit einer Simulation (Simulink-Modell *Regelkreis_ZK.slx* bei Sprüngen des PWM-Wertes von 220 auf 180 und einem Wert von $L = 2mH$).

Abbildung 14: Sprungantwort des Motorstroms i_M

Das Messergebnis zeigt einige Auffälligkeiten:

- Die elektrische Zeitkonstante ergibt sich zu $T_{el,m} \approx \frac{2\text{mH}}{11,9\Omega} \approx 170\mu\text{sec}$. Sie liegt damit entgegen der obigen Annahme in der Größe der Zeitkonstanten der Anpassschaltung (Abb. 13).
- Der Stromverlauf zeigt hochfrequente Schwingungen. Dieser rippelige Stromverlauf ergibt sich, weil der Übergangswiderstand am Kommutator vom Drehwinkel der Welle abhängt.
- Der Strom fällt nach dem Sprung wieder ab. Das liegt daran, dass sich entgegen unserer obigen Annahme die Drehzahl bereits kurz nach dem Anstieg des Stroms verändert und die induzierte Spannung u_{EMK} damit nicht konstant ist.

Es macht daher keinen großen Sinn, die Messung noch genauer zu machen. Wir arbeiten deshalb mit $L = 2\text{mH}$ weiter.

4.2.2 Reibung

Um die Reibung zu bestimmen, kann man die aufgenommene elektrische Leistung $u_{motor,in} \cdot i_M$ im Leerlauf bei verschiedenen Drehzahlen n messen. Diese „Leerlauf“leistung wird im Wesentlichen benötigt, um die Reibung zu überwinden. Aus der Leerlaufleistung lässt sich dann das Reibmoment M_R und der Reibbeiwert d berechnen, wie bereits in Vorbereitungsaufgabe 2 beschrieben.

Die Messergebnisse in Abbildung 15 zeigen allerdings, dass wir nicht nur viskose Gleitreibung, d.h. rein drehzahlproportionale Reibung haben (blaue Kurve). Die realen Messdaten (rote Kurve) zeigen einen Versatz zwischen der Geraden bei positiven Drehzahlen und der Geraden bei negativen Drehzahlen. Bei sehr kleinen Drehzahlen um 0 dominiert die Haftreibung (Coulombsche Reibung). Für die Drehzahlregelung bei höheren Drehzahlen werden wir trotzdem von reiner Gleitreibung ausgehen (blaue Kurve). Bei der Positionsregelung aber sind durch die stark nichtlineare Haftreibung einige Probleme zu erwarten.

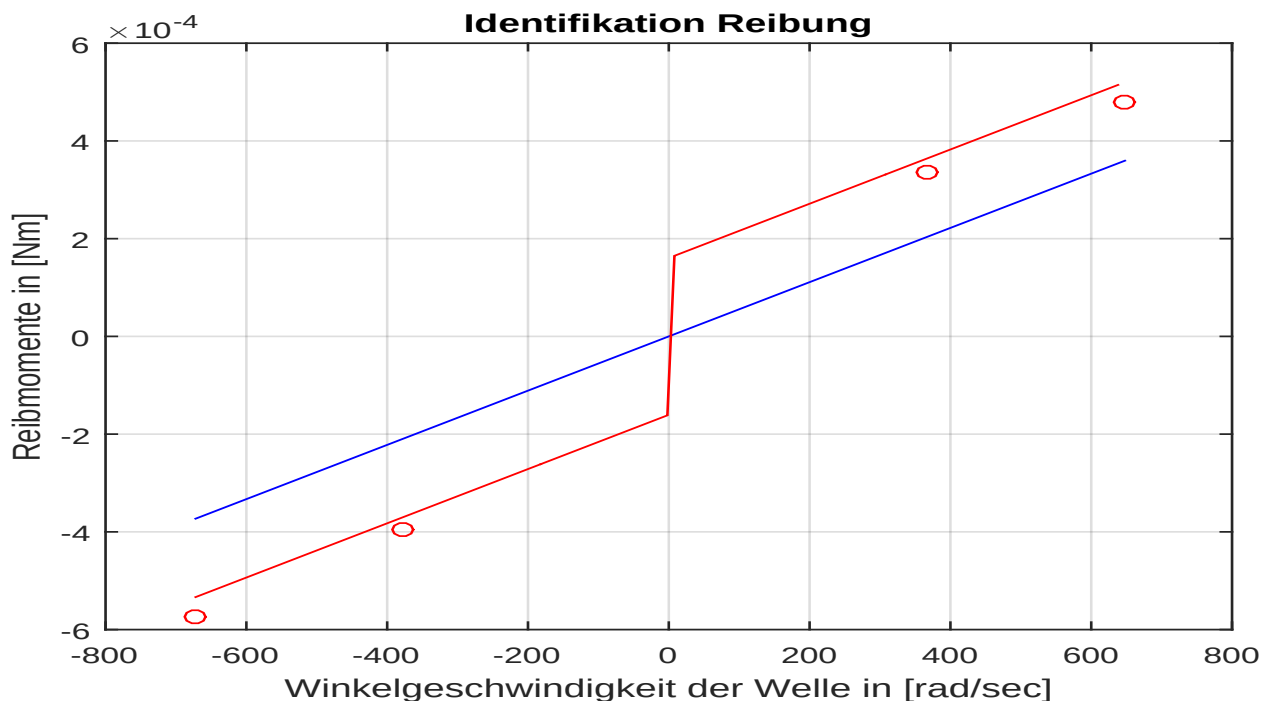


Abbildung 15: Ergebnisse der Reibungsmessung (rot: Messdaten, blau: rein viskose Reibung)

4.2.3 Trägheitsmoment

Zur Identifikation des Trägheitsmoments J gehen wir vor wie bei der Messung der Induktivität L (Abschnitt 4.2.1). Wir messen die Sprungantwort des Motorstroms i_M bei Sprüngen der Eingangsspannung u_{PWM} , indem wir den PWM-Wert zwischen 220 und 180 umschalten. Dieses Mal sehen wir uns allerdings einen wesentlich längeren Zeitbereich an. Wie sich in Abbildung 14 schon

angedeutet hat, fällt der Strom nach dem Anstieg wieder ab, weil sich die Drehzahl und damit die induzierte Spannung u_{EMK} verändert. Das Trägheitsmoment J bestimmt dabei, wie schnell sich die Drehzahl ändert. Abbildung 16 zeigt den vollständigen Verlauf. Der gemessene Stromverlauf ist in rot dargestellt. Überlagert ist in blau das Ergebnis einer entsprechenden Simulation mit dem Simulink-Modell *Regelkreis_ZK.slx*. Dabei wurde J in der Simulation so lange verändert, bis der simulierte Verlauf von i_M mit dem gemessenen Verlauf bestmöglich übereingestimmt hat. Für das kleinere Trägheitsmoment haben wir $J_{K(lein)} = 0,7 \cdot 10^{-6} \text{ Nmsec}^2$ gefunden.

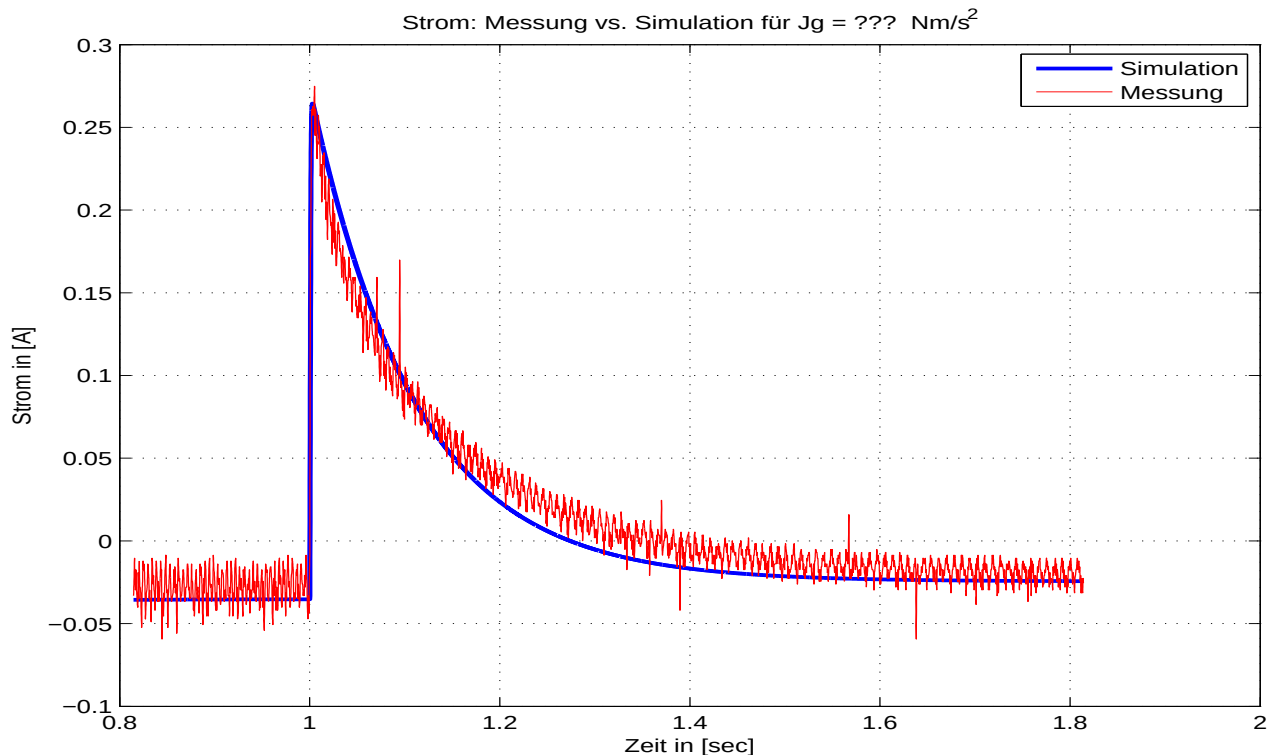


Abbildung 16: Ergebnisse der Messung des Trägheitsmoments (rot: Messdaten, blau: Simulation)

Vorbereitungsaufgabe 7: Identifikation des großen Trägheitsmoments $J_{G(ross)}$

Führen Sie die beschriebene Simulation mit dem Simulink-Modell *Regelkreis_ZK.slx* durch. Vergessen Sie nicht, im Modell auf den PWM-Eingang umzuschalten. Um das Modell zu parametrieren, müssen Sie wieder das M-Skript *params_m.m* ausführen. Dort wird bisher $J = J_{K(lein)}$ verwendet. Die Messung in Abbildung 16 wurden aber mit $J_{G(ross)}$ durchgeführt. Welcher Wert für $J_{G(ross)}$ liefert in Ihrer Simulation annähernd denselben Verlauf wie im Bild?

$J_{G(ross)} =$

Stellen Sie in *params_m.m* anschließend wieder auf das kleinere Trägheitsmoment $J = J_{K(lein)}$ zurück.

4.2.4 Motorkonstante k_M

Als letzte Größe des Gleichstrommotors bestimmen wir die Motorkonstante k_m . Dazu verwenden wir folgendes „Experiment“:

- Wir klemmen den Motor vom Leistungsverstärker ab, d.h. $i_M = 0$ (vergl. Abb. 1) und bringen ihn von außen auf eine konstante Drehzahl, indem wir ihn über den Generator antreiben.
- Mit einem Digitalvoltmeter messen wir dann die Spannung an den Klemmen des Motors. Da kein Strom fließt, ist die Klemmenspannung gleich u_{EMK} .
- Gleichzeitig ist nach dem Blockschaltbild in Abb. 10 $u_{EMK} = k_m \cdot \omega = k_m \cdot 2\pi n$.
 $\Rightarrow k_m = \left| \frac{u_{EMK}}{2\pi n} \right|$ (Achtung: u_{EMK} in V, n in $\frac{U}{sec}$ nicht $\frac{U}{min}$!).

Laboraufgabe 1: Bestimmung der Motorkonstante k_m

Führen Sie die oben beschriebene Messung in folgenden Schritten durch:

- Entfernen Sie die Steckbrücke SB zwischen Generator und Lampe (vergl. Abb. 3).
- Entfernen Sie die Verbindung vom Leistungsverstärker zum Motor und schließen Sie stattdessen den Leistungsverstärker an den Generator an.
- Messen Sie mit einem Digitalvoltmeter die Klemmenspannung U_{EMK} des Motors.
- Messen Sie mit dem Digitalspeicheroszilloskop DSO die TTL-Signale des Positionsgebers und deren Periodendauer T_{TTL} (vergl. Abb. 4). Da der Geber 100 Pulse je Umdrehung liefert, ist die Motordrehzahl $n_{TTL} = \frac{60}{100 \cdot T_{TTL}}$, Ergebnis in $\frac{U}{min}$ mit T_{TTL} in sec. Die Formel liefert kein Vorzeichen für die Drehzahl. Wählen Sie das Vorzeichen von n_{TTL} so, dass es dasselbe ist wie das Vorzeichen von U_{EMK} .
- Laden Sie das Codewarrior-Projekt auf das Dragon12-Board und führen Sie es aus. Wählen Sie über die Bedientaster den Handbetrieb (manual), stellen Sie über die Taster die PWM-Werte nach Tabelle 4 ein und tragen Sie die Messdaten ein.

In der ersten Zeile des LCD-Displays wird in diesem Betriebsmodus die vom Mikrocontroller über den Tachogenerator analog gemessene Drehzahl angezeigt. Tragen Sie diese Werte in die Zeile $n_{\mu C, ADC}$ in der Tabelle ein.

Falls die von Ihnen berechneten Werte für n_{TTL} um mehr als ca. $\pm 10\%$ von den Werten für $n_{\mu C, ADC}$ abweichen, überprüfen Sie Ihre Berechnung. Möglicherweise haben Sie einen Fehler mit den Einheiten gemacht.

- Berechnen Sie aus den Messergebnissen von n_{TTL} (!) und U_{EMK} den Mittelwert von k_m (Einheit nicht vergessen!). Vergleichen Sie Ihr Ergebnis mit dem Wert aus der Simulationsdatei *params.m.m*. Bei einer Abweichung von mehr als ca. $\pm 20\%$ haben Sie sich möglicherweise verrechnet oder bei den Einheiten vertan.

PWM-Wert:	220	180	75	35	Einheit
T_{TTL}					μsec
U_{EMK}					V
$n_{\mu C, ADC}$					U/min
n_{TTL}					U/min
k_m					?

Ergebnis: Mittelwert von $k_m =$

Tabelle 4: Messung Motorkonstante k_m und Überprüfung des Tachogenerators

Im weiteren Verlauf des Labors soll der Motor untersucht werden. Daher müssen Sie den **Leistungsverstärker wieder an den Motor anschließen**.

4.3 Validierung des Simulationsmodells, Drehzahlmessgenauigkeit

Nachdem nun alle Parameter des Modells mit Hilfe von Messungen identifiziert wurden, ist die Modellbildung abgeschlossen. Um sicherzustellen, dass Modell und reales System übereinstimmen, sollten aber noch einige weitere Messungen und Simulationen miteinander verglichen werden. So zeigt Abb. 17 den Vergleich des Drehzahlverlaufs zwischen Simulation und Messung bei einem Sprung der PWM-Ansteuerung $180 \rightarrow 220$. Während der zeitliche Verlauf recht gut übereinstimmt, gibt es offensichtlich einen Drehzahlfehler, der zudem von der Höhe der Drehzahl abhängt.

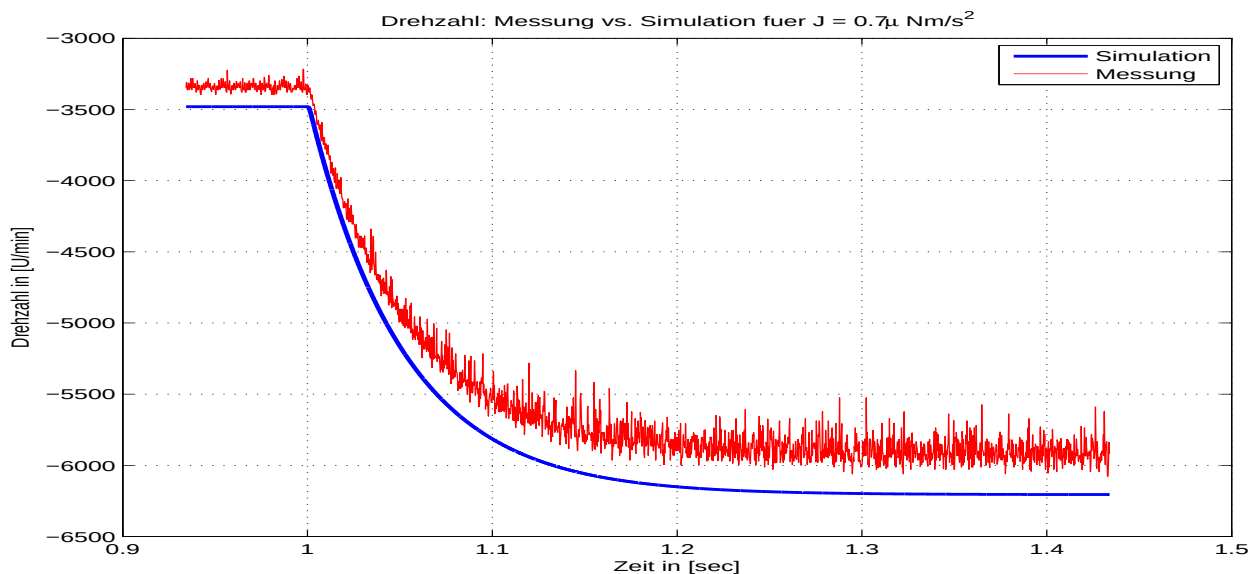


Abbildung 17: Validierung des Modells gegen Messung: Gemessener Drehzahlverlauf $n_{\mu C, ADC}$ (rot) bzw. simulierter Drehzahlverlauf (blau) bei PWM-Ansteuerung 180 $\rightarrow$ 220.

Einer der Hauptunterschiede zwischen Simulation, den Messungen zur Parameteridentifikation und zur Regelung mit dem Mikroprozessor ist die Drehzahlerfassung. Wenn wir bei den Messungen die Drehzahl benötigt haben, haben wir diese über die Periodendauer der digitalen TTL-Signale des Positionssensors bestimmt, die wir mit dem DSO gemessen haben. Diese Messung ist sehr genau. Der Regler im Mikroprozessor dagegen verwendet das analoge „Drehzahlsignal“ des Tachogenerators. Dabei hat sowohl der Tachogenerator selbst eine Toleranz, als auch insbesondere die Anpassschaltung zwischen Tachogenerator und ADC-Eingang des Mikrocontrollers. Wir überprüfen daher die Drehzahlerfassung:

Laboraufgabe 2: Überprüfung der analogen Drehzahlerfassung

Bei der vorigen Laboraufgabe haben Sie in Tabelle 4 sowohl die aus den digitalen TTL-Signalen des Positionssensors ermittelte Drehzahl n_{TTL} als auch die vom Mikroprozessor über den analogen Tachogenerator gemessene Drehzahl $n_{\mu C, ADC}$ erfasst.

- Zeichnen Sie ein Diagramm $n_{\mu C, ADC}(n_{TTL})$ und versuchen Sie, eine Ausgleichsgerade zu bestimmen. Theoretisch müsste die Kennlinie exakt durch den Ursprung gehen und eine Steigung von 1 haben. Bestimmen Sie mit der Ausgleichsgerade die Messfehler:

Offset bei Drehzahl 0 in U/min: $n_{ofs} =$

Steigung k_n :

Sie können für dieses Diagramm das Matlab-Skript `drehzahlMessung.m` verwenden.

Editieren Sie das Skript und tragen Sie Ihre Messwerte ein, bevor Sie das Skript ausführen. Das Skript berechnet den Offset und die Steigung der Ausgleichsgerade mit Hilfe der Matlab-Funktion `polyfit()` und zeichnet die Ausgleichsgerade mit Hilfe der Matlab-Funktion `polyval()`.

- Nehmen Sie dieses Diagramm in Ihren Laborbericht auf.

Regelungen können nicht genauer sein als die verwendeten Sensoren, selbst wenn die Regelung theoretisch keine bleibende Regelabweichung hat. Wenn die Genauigkeit des Sensors nicht ausreicht, könnte man z.B. wie hier die Sensorkennlinie ausmessen und den Sensorfehler im Reglerprogramm durch eine Korrektur berücksichtigen. Alternativ kann man einen anderen Sensor verwenden. Im vorliegenden Fall wäre es naheliegend, das Mikroprozessor-Programm so ändern, dass die Drehzahl aus den digitalen TTL-Signalen ermittelt wird. Dies wäre durchaus möglich, da die Positionssignale ohnehin an den Capture-Eingängen der Timer-Einheit des Mikrocontrollers angeschlossen sind. Problematisch ist die Messung allerdings bei sehr kleinen Drehzahlen. Wenn die Abstände der Signalfanken größer sind als die Abtastzeit des Drehzahlreglers, gibt es praktisch keinen Drehzahlmesswert mehr. Der Tachogenerator dagegen liefert selbst bei kleinen Drehzahlen noch eine Information, solange seine Signalspannung nicht kleiner wird als die Quantisierungsstufen des ADC. In der Praxis kombiniert man oft beide Ansätze oder setzt Inkrementalgeber mit wesentlich mehr Pulsen je Umdrehung ein.

5 Auslegung des Drehzahlreglers in der Simulation

Bitte stellen Sie alle Parameter, die Sie im Verlauf des Laborversuchs in der Datei *params.m.m* verändert haben, wieder auf die originalen Werte zurück. Das größere Trägheitsmoment J_g können Sie auf dem Wert belassen, den Sie ermittelt haben. Bitte verwenden Sie aber für die Reglerauslegung das kleinere Trägheitsmoment $J = J_k$.

5.1 Dimensionierung des PI–Drehzahlreglers

Der Drehzahlregler ist mit Hilfe des Matlab–Skripts *auslegung_DrehzahlRegler_PI.m.m* auszulegen. Der offene Regelkreis setzt sich zusammen aus:

- Der Übertragungsfunktion k_{cond_pwm} nach Abb. 7 für die PWM–Ansteuerung vom Eingang *PWM* bis zum Ausgang $U_{Motor,in}$
- Der Streckenübertragungsfunktion $G_{Strecke_n}$ der Drehzahlregelstrecke nach Abb. 10 vom Eingang $U_{Motor,in}$ bis zum Ausgang n (ohne den Generator für das Belastungsmoment). Die Strecke setzt sich aus dem elektrischen und dem mechanischen Teil zusammen.
- Dem Tachogenerator mit dem Verstärkungsfaktor k_{cond_n} nach Abb. 8 vom Eingang n bis zum Eingang des ADC.

Als Regler verwenden wir einen PI–Regler, den wir mit Polkompensation und Vorgabe der Phasenreserve (Nyquist–Verfahren) auslegen. Bei der Polkompensation wird die langsamste Zeitkonstante der Regelstrecke durch die Reglernullstelle kompensiert.

Vorbereitungsaufgabe 8: Dimensionierung des Drehzahlreglers

Sehen Sie sich das M–File *auslegung_DrehzahlRegler_PI.m.m* an und versuchen Sie den Inhalt zu verstehen.

Dimensionieren Sie die Verstärkung k_{Rn} des Drehzahlreglers (für $J = J_k$) mit Hilfe des Matlab–Skripts so, dass der **offene** Regelkreis eine Phasenreserve von $\varphi_R = 60^\circ$ hat. Die Nachstellzeit T_n wird im Matlab–Skript automatisch so eingestellt, dass die langsamste Streckenzeitkonstante kompensiert wird. Das Bodediagramm des offenen Regelkreises wird durch die Matlab–Funktion *margin()* gezeichnet und die Phasenreserve als *Pm* in das Diagramm eingetragen. Verändern Sie die Verstärkung im Matlab–Skript solange, bis Sie die geforderte Phasenreserve erhalten.

Welche Werte ergeben sich für die Proportionalverstärkung, die Nachstellzeit und die Durchtrittskreisfrequenz des Drehzahlreglers?

$$k_{Rn} =$$

$$T_n =$$

$$\omega_{D,n} =$$

Tragen Sie diese Werte k_{Rn} und T_n auch in die Parameter–Initialisierung *params.m.m*

ein. Speichern Sie dieses Skript und führen Sie es aus, damit die Werte auch bei der nachfolgenden Simulation des Modells in Simulink zur Verfügung stehen.

Begrenzung der Stellgröße des Drehzahlreglers:

Wie aus der Vorlesung bekannt, ist die Stellgröße eines realen Reglers immer beschränkt. Die Ansteuerung des PWM-Ausgangs des Mikrocontrollers kann nur mit Werten von $0 \dots 255$ erfolgen. Daher ist ein entsprechendes Sättigungselement im Simulink-Modell vorgesehen. Der Ausgang des Reglers besitzt damit einen Wertebereich von $-127,5 \dots +127,5$, weil nachfolgend der Arbeitspunkt durch einen Offset von $127,5$ eingestellt wird (siehe Simulink-Modell *Regelkreis_ZK.slx*).

Weil der Drehzahlregler ein PI-Regler ist, werden wir in der Praxis bei großen Sollwertsprüngen ein schlechtes Einschwingverhalten bekommen, sobald der Regelkreis dynamisch in die Begrenzung geht (vergl. Regelungstechnik 1 - Laborversuch Regler). Wir müssen für den Drehzahlregler daher eine entsprechende **Anti-Windup-Maßnahme** mit den entsprechenden Grenzen einbauen.

5.2 Verifikation des Drehzahlreglers in der Simulation

Der entworfene Drehzahlregler soll nun in der Simulation verifiziert werden. Die Parameter des Simulink-Modells *Regelkreis_ZK.slx* sind im M-File *params_m.m* angegeben. Bevor Sie simulieren, sollten Sie dieses Skript ausführen. Stellen Sie die Schalter und den Step-Block für den Sollwert des Drehzahlreglers im Simulink-Modell so ein, dass der Drehzahlregler mit den gewünschten Sollwertsprüngen entsteht. Sie sollten jeweils bis ca. $t_e = 2.5\text{sec}$ simulieren.

Vorbereitungsaufgabe 9: Führungssprungantwort

Vorbereitungsaufgabe 9.1 Kleinsignal

Simulieren Sie einen Sollwertsprung von 3000 auf $3100\text{U}/\text{min}$ ohne Störung zur Zeit $t_w = 1\text{sec}$. Die Ergebnisse sollten ungefähr so aussehen wie der Führungssprung aus der vorherigen Aufgabe, als Sie den Drehzahlregler ausgelegt haben. Fügen Sie die Zeitverläufe von Drehzahl und Strom in Ihren Bericht ein. **Achten Sie auf eine sinnvolle Skalierung der Achsen, damit man den interessierenden Teil der Zeitverläufe auch sehen kann. Die interessierende Sprungantwort findet im Zeitintervall zwischen 1sec und ca. 1.1sec statt!!**

Vorbereitungsaufgabe 9.2 Großsignal

Simulieren Sie

- (a) einen Sollwertsprung von 1000 auf $3000\text{U}/\text{min}$ sowie
- (b) einen Sollwertsprung von 1000 auf $6000\text{U}/\text{min}$

jeweils zur Zeit $t_w = 1\text{sec}$ ohne Störung. Fügen Sie die beiden Zeitverläufe von Drehzahl und Strom in Ihren Bericht ein. **Achten Sie auch hier auf sinnvolle Skalierung der Achsen!**

6 Implementierung des PI–Reglers auf dem Mikrocontroller

Um den Drehzahlregler auf das reale Dragon12–Mikrocontrollerboard zu portieren, sind drei Schritte notwendig.

6.1 Umwandlung in einen zeitdiskreten PI–Algorithmus

Wie bereits in Regelungstechnik 1 besprochen, kann ein zeitkontinuierlicher PI–Regler

$$G_R(s) = \frac{U(s)}{E(s)} = k_R \cdot \left(1 + \frac{1}{sT_n}\right)$$

durch folgenden rekursiven Algorithmus im Zeitbereich nachgebildet werden, der periodisch im Abtastraster berechnet werden muss:

$$u_I(k) = u_{I,alt} + k_R \cdot \frac{T}{T_n} \cdot e(k)$$

$$u(k) = k_R \cdot e(k) + u_I(k)$$

Dabei ist T die Abtastzeit und es wird angenommen, dass die Abtastzeit deutlich kleiner ist als die Nachstellzeit T_n , d.h. es wird ein quasi–zeitkontinuierlicher Entwurf vorausgesetzt. Die erste Zeile berechnet den I–Anteil aus dem I–Anteil des letzten Zeitschritts und dem aktuellen Inkrement, die zweite Zeile addiert dazu den P–Anteil. In einer C–Funktion muss die Variable $u_{I,alt}$ persistent implementiert werden, z.B. als statische lokale Variable, und auf z.B. den Wert 0 initialisiert werden.

Die Anti–Windup–Begrenzung sowie die Stellgrößenbeschränkung mit den Grenzwerten u_{min} und u_{max} ist wie folgt möglich:

```
if(u > u_max) {
    u = u_max;          /* u_I_alt bleibt */
}else if(u < u_min) {
    u = u_min;          /* u_I_alt bleibt */
}else{
    u_I_alt = u_I;      /* Index Shift */
}
```

6.2 Test des zeitdiskreten PI–Algorithmus in C im Simulink–Modell

Vorbereitungsaufgabe 10: Implementieren des PI–Reglers in C und Test in Simulink

Das Simulink–Modell *Regelkreis_ZD_C.slx* enthält einen C/C++–Block (Simulink S–Function), in dem die C–Funktion *zPlawu_n.c* verwendet wird. Ergänzen Sie diese Funktion so, dass sie den geforderten Anti–Windup–PI–Regler implementiert. Die Abtastzeit sowie die Reglerparameter werden der S–Function in der Maske des Reglerblocks im Simulink–Modell übergeben.

Vergessen Sie nicht, die C-Funktion mit *mex zPlawu_n.c* in eine ausführbare Datei zu übersetzen, bevor Sie das Simulink-Modell simulieren. Zusätzlich müssen Sie auch die C-Funktion für den Positionsregler mit *mex zP_pos.c* entsprechend übersetzen. Simulieren Sie Ihren Regler bei der voreingestellten Abtastzeit. Sie sollten bei den verschiedenen Sprungantworten (3000 auf 3100 U/min, 1000 auf 3000 U/min und 1000 auf 6000 U/min) praktisch dieselben Ergebnisse bekommen, wie im letzten Abschnitt.

Zu dieser Vorbereitungsaufgabe müssen Sie nichts abgeben.

6.3 Portieren des zeitdiskreten PI-Algorithmus auf das Dragon12-Board

Damit sind alle Vorarbeiten abgeschlossen und der Reglerentwurf kann auf dem Mikrocontroller implementiert werden.

Laboraufgabe 3: Portieren und Testen des Reglers auf dem Dragon12-Board

Öffnen Sie das Codewarrior-Projekt *Systemtechnik.mcp* im Verzeichnis *uC*. Die Source-Dateien, die dieses Projekt verwendet, liegen immer im Verzeichnis *Sources*.

Implementieren Sie den Drehzahlregler. Sie müssen dazu im Codewarrior-Projekt in der Datei *main.c* in der Funktion *rpmControl()* den entsprechenden C-Code ergänzen und die Reglerparameter eintragen. Diese Funktion wird durch die ISR *myADC1Callback* im Abtastraster zyklisch aufgerufen. Verwenden Sie die C-Datei, die Sie in der letzten Vorbereitungsaufgabe entwickelt haben, als Vorlage:

- Die Variablen, die Sie beim Implementieren der Regler verwenden sollen, sind im Quelltext im Kommentar beschrieben. **Lesen Sie diesen Kommentar!**
- Der PI-Regler soll wieder mit einer **Anti-Windup**-Maßnahme ausgestattet sein.
- Die Abtastzeit des Regler (= Periode des RTI-Interrupts) ist auf $1536\mu\text{sec}$ eingestellt.
- Stellen Sie die Solldrehzahlen für den Drehzahlregler im Automatik-Betrieb in *main.h* so ein, dass dieser zwischen den Solldrehzahlen 1000 und 3000 U/min hin und herschaltet.

Wenn Ihr Programm fehlerfrei compiliert wurde, laden Sie das Codewarrior-Projekt auf den Mikrocontroller. Wenn der Regler funktioniert, sollten Sie dies hören. Wenn die **Dioden der Vierquadrant-Anzeige wild blinken**, schalten Sie das Board bitte möglichst **schnell aus** und debuggen Sie Ihre Implementierung.

Messen Sie den Drehzahlverlauf mit Hilfe des Softwareoszilloskops (sh. S.35). Fügen Sie die Sprungantworten in Ihren Laborbericht ein. Zoomen Sie dabei bitte so in die Diagramme hinein, dass einmal die ansteigende und einmal die abfallende Flanke der Drehzahl bildfüllend zu sehen ist. Achten Sie auf einen möglichst einheitlichen Maßstab bei allen Diagrammen.

7 Positionsregler

7.1 Dimensionierung und Test des Positionsreglers mit Matlab/Simulink

Als Positionsregler verwenden wir einen P–Regler, den wir mit Vorgabe der Phasenreserve (Nyquist–Verfahren) auslegen.

Der Regler wird mit Hilfe des M–Files *auslegung_Positionsregler_P.m* dimensioniert. Im Matlab–Skript wird zunächst die Streckenübertragungsfunktion der Positionsregelstrecke bestimmt. Darüber hinaus wird die Übertragungsfunktion der Rückführung (Positionsdeko­der als P–Glie­d modelliert) angegeben. Die übrige Vorgehensweise im Matlab–Skript entspricht derjenigen bei der Auslegung des Drehzahlreglers oben.

Vorbereitungsaufgabe 11: Dimensionierung und Simulation des Positionsreglers

Vorbereitungsaufgabe 11.1 Dimensionierung

Damit der Positionsregler praktisch kein Überspringen zeigt, dimensionieren Sie den Positionsregler mit Hilfe des Matlab–Skripts so, dass sich eine Phasenreserve von $\varphi_R = 80^\circ$ einstellt. Sehen Sie sich das M–File an und versuchen Sie seinen Inhalt zu verstehen, bevor Sie das Skript verwenden. **Stellen Sie sicher, dass Sie in die Datei *params.m* die Werte k_{Rn} und T_n eingetragen haben, die sich bei der Dimensionierung des Drehzahlreglers ergeben haben, bevor Sie den Positionsregler auslegen!**

Welche Werte ergeben sich für den Proportionalbeiwert und die Durchtrittskreisfrequenz des Positionsreglers?

$$k_{Rpos} =$$

$$\omega_{D,pos} =$$

Tragen Sie diesen Wert von k_{Rpos} in das Matlab–Skript *auslegung_Positionsregler_P.m* ein und verifizieren Sie Ihre Dimensionierung.

Wenn das System das gewünschte Führungsverhalten zeigt, tragen Sie den Wert für k_{Rpos} auch in die Parameter–Initialisierungsdatei *params.m* ein.

Vorbereitungsaufgabe 11.2 Simulation

Der Positionsregler soll nun in der Simulink–Simulation verifiziert werden. Verwenden Sie dazu das Simulink–Modell *Regelkreis_ZK.slx*.

Begrenzung der Stellgröße des Positionsreglers:

Um zu vermeiden, dass der unterlagerte Drehzahlregler mit einem realitätsfernen Sollwert angesteuert wird, wird der Ausgang des überlagerten Positionsreglers (= Sollwert des Drehzahlreglers) im Simulink–Modell auf sinnvolle Drehzahlwerte begrenzt. Die Begrenzung erfolgt so, dass sich ein maximaler Drehzahlsollwert von $\pm 7800 U/min$ ergibt.

Stellen Sie die Schalter und den Step–Block für den Sollwert des Positionsreglers im Simulink–Modell so ein, dass der Positionsregler einen Sollwertsprung von 0 auf $10U$ ($U = \text{Umdrehungen}$) zur Zeit $t_w = 1 \text{ sec}$ ohne Störgröße ausführt. Die Simulationdauer soll ca. 2.5sec betragen.

Fügen Sie die Zeitverläufe von Position und Drehzahl in Ihren Laborbericht ein.

7.2 Implementierung der Positionsregelung auf dem Dragon12-Board

Laboraufgabe 4: Überprüfen der Drehzahlregelung und Einbau der Positionsregelung

Bauen Sie den **P-Positionsregler** in Ihr Programm ein. Die entsprechende Stelle in der Funktion *positionControl()* in der Datei *main.c* sowie die Variablen, die Sie lesen bzw. schreiben dürfen, sind entsprechend kommentiert.

Laden Sie das Programm auf das Dragon12-Board und testen Sie nochmals die **Drehzahlregelung** mit den Drehzahlsprüngen wie oben. Die Drehzahlverläufe dürfen sich gegenüber der vorigen Messung nicht verändert haben.

Schalten Sie nun über die Bedientasten auf die **Positionsregelung** um und stellen Sie den Automatik-Betrieb ein, bei dem Sollpositionssprünge zwischen 0 und 10 Umdrehungen gemacht werden. Der schnelle Wechsel zwischen den beiden Positionen sollte an der Welle des Motors sichtbar (und hörbar!) sein. **Falls die Dioden der Vierquadrantenanzeige wild blinken, schalten Sie das Board bitte möglichst schnell wieder aus** und überprüfen Sie Ihre Implementierung.

Ihr Laborbericht sollte den vollständigen C-Programmcode des Positions- und des Drehzahlreglers einschließlich der zugehörigen Variablen- und Konstantendefinitionen enthalten. Der restliche Programmcode ist nicht erforderlich.

Damit ist der erste Laborversuch beendet. Die ausführliche Analyse der Positionsregelung folgt im zweiten Laborversuch.

8 Anhang: Analyse von Messdaten

8.1 Software-Oszilloskop

Interne Werte des Mikrocontrollerprogramms, z.B. die mit dem ADC gemessene Motordrehzahl, die Solldrehzahl oder die Istposition, können über die serielle Schnittstelle im Echtzeitbetrieb an den PC übertragen und in Matlab angezeigt werden. Dazu wird **im Mikrocontrollerprogramm** am Ende der Interrupt-Service-Routine `myADC1Callback()`, die im RTI-Abtastraster alle $T = 1536\mu\text{sec}$ ausgeführt wird, die Funktion

```
SoftScope((int16) set_rpm, (int16) measured_rpm);
```

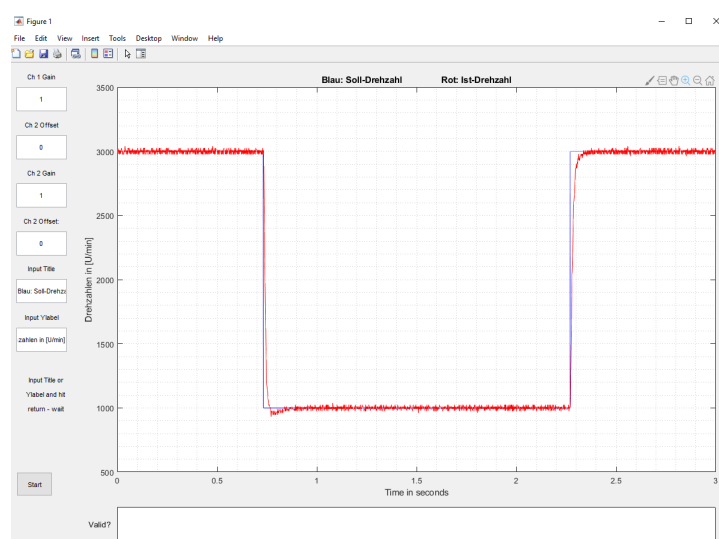
aufgerufen. Beachten Sie, dass die übertragenen Werte in 16bit Integerzahlen konvertiert werden. Für eine Übertragung von Gleitkommazahlen oder von mehr als zwei Werten innerhalb der Abtastzeit ist die serielle Schnittstelle (115kbit/sec) zu langsam.

Das Dragon12-Mikrocontrollerboard muss zusätzlich zur Debugger-Schnittstelle über die zweite serielle Schnittstelle mit dem PC verbunden werden. **In der Matlab-Konsole** muss die folgende Funktion für das Software-Oszilloskop ausgeführt werden:

```
SoftScope(comIF, ...)
```

Als erster Parameter muss die Nummer der seriellen Schnittstelle, z.B. `comIF=2`, übergeben werden. Falls Sie die Nummer nicht kennen, können Sie `comIF=0` setzen, dann werden alle vorhandenen Schnittstellen aufgelistet. Die dargestellten Zeitsignale sind die vom Mikrocontroller-Programm gesendeten 16bit-Integerwerte. Über die Bedienoberfläche des Programmfensters bzw. weitere Aufrufparameter (Gain und Offset) der Funktion (Hilfe mit `help SoftScope`) können Sie die Werte umskalieren, z.B. in U/min .

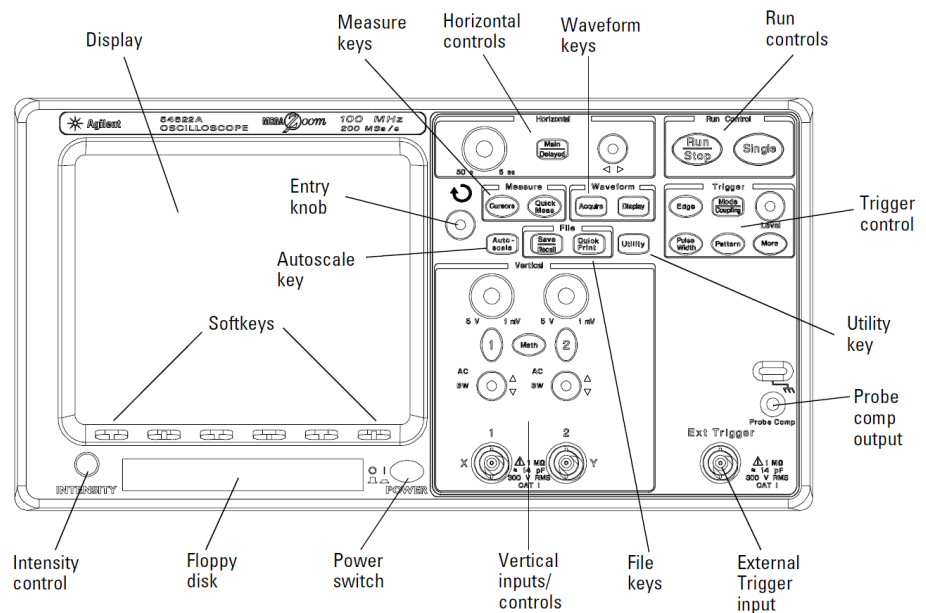
Das Bildschirmfenster zeigt einen Zeitbereich von einigen Sekunden an und wird laufend aufgefrischt. Um Details näher analysieren zu können, stoppen Sie das Oszilloskop über den Start/Stop-Button im Programmfenster und zoomen Sie im Bild mit den üblichen Matlab-Werkzeugen.



Achtung: Falls das Software-Oszilloskop hängen bleibt, schließen Sie das Fenster und starten Sie es neu. Eventuell müssen Sie anschließend `clear all` ausführen oder sogar Matlab neu starten.

8.2 Bedienung des Digital-Speicher-Oszilloskops

Die Bedienung erfolgt über Tasten (**Hardkeys**) und Drehknöpfe (**Knobs**) auf der Vorderseite des Gerätes sowie Menütasten (**Softkeys**) unterhalb des Displays. Wenn eine Menütaste mehrere Optionen hat, wählt man diese über den Drehknopf **Entry knob**.



Grundeinstellungen:

- **Tastköpfe anschließen** und mit Messsignalen verbinden. Zugehörige Kanäle über Hardkeys 1 und 2 einschalten.
- Hardkey **Autoscale** drücken: Stellt Kanalverstärkung und Trigger automatisch ein, funktioniert aber nur bei „schnellen“ Signalen, z.B. den TTL-Signalen des Positionsgebers. Für Messung von Sollwertsprüngen der Drehzahl mit dem Tachogebersignal nicht geeignet.
- Erneutes Drücken von Autoscale schaltet auf die vorige manuelle Einstellung zurück.

Kanalverstärkung und Zeitablenkung:

- Die **Verstärkung** kann für jeden Kanal an den großen Drehknöpfen im Bedienfeld Vertical eingestellt werden. Mit den darunterliegenden kleinen Drehknöpfen kann für jeden Kanal der **Offset**, d.h. die vertikale Lage, verstellt werden.
- Die **Zeitablenkung** wird mit dem großen Drehknopf im Bedienfeld Horizontal eingestellt.
- Die eingestellten Werte (V/Div bzw. s/Div) werden im Display in der oberen Zeile angezeigt.

Trigger:

- Hardkey **Edge** wählt man das Menü im Display aus. Mit Softkey **Rising** ↑ bzw. **Falling** ↓ kann die steigende oder fallende Signalfanke zum Triggern ausgewählt werden. Mit den Softkeys 1 bzw. 2 wählt man den **Triggerkanal** aus. Der **Triggerpegel** kann über den Drehknopf Level im Bedienfeld Trigger eingestellt werden.
- Hardkey **Mode** wählt das Menü im Display aus. Über den Softkey Mode wählt man Auto oder Normal (für langsame Signale wie Sollwertsprünge besser geeignet). Bei **Coupling** sollte DC ausgewählt werden.

9 Anhang: Bedienoberfläche des HCS12-Programms

Bedientasten:

- Taste SW2 (ganz links):
Umschalten zwischen Drehzahlregelung (R), Positionsregelung (P) und Handbetrieb (M)
- Taste SW3:
Umschalten zwischen automatischer Sollwerterzeugung (eingebauter Rechteckgenerator) oder Sollwertvorgabe über SW4/SW5.
- Taste SW4:
Sollwert vergrößern
- Taste SW5 (ganz rechts):
Sollwert verringern

Anzeige im LCD-Display obere Zeile:

- Im RPM-(Drehzahlregler)-Modus:
R: w =Solldrehzahl e =Regelfehler (beide in U/min)
- Im Positionsregler-Modus:
P: w =Sollposition e =Regelfehler
- Im Manual-(Hand)-Modus:
M: u =manuell vorgegebener PWM-Wert n =gemessene Drehzahl (in U/min)

Anzeige im LCD-Display untere Zeile:

- Bedienhinweise zu den Tasten SW4, SW5 / Fehlermeldung / Drehzahl aus TTL-Signalen

LEDs (am Port B, Ansteuerung siehe Programmcode):

- LED7 (ganz links): Blinkt ungefähr im Sekundentakt
- LED6: Blinkt mit Abtastfrequenz (= wirkt optisch wie dauernd an)
- LED5: Im Abtastraster eingeschaltet, solange Regleralgorithmus berechnet wird (= wirkt optisch wie dauernd an). Für Laufzeitmessungen der Rechenzeit mit dem Oszilloskop.
- LED4: Im Abtastraster von Beginn bis Ende der seriellen Datenübertragung (SoftScope) eingeschaltet. Für Zeitmessungen mit dem Oszilloskop.
- LED3: Fehler in der Drehrichtungserkennung (Plausibilität Tachosignal - Positionsgeber). Gelbes und graues Kabel des Positionsgebers vertauscht?
- LED1: Blinkt im Takt des Sollwertgenerators, kann zum Triggern bei Sollwertsprüngen verwendet werden.
- LED0 (ganz rechts): Ansteuerung eines Relais auf dem Motorboard, im Normalbetrieb dauernd an. Nicht ändern!

10 Anhang: Anschlüsse Motorboard - Mikrocontroller

	Motorboard bzw. Anpassschaltung	Mikrocontroller
PWM–Ansteuerung	A: PWM–Eingang (schwarzes Kabel mit Schirm)	PP0 (PWM-Ausgang) Schirm (Masse) auf GND PWM-Kanal 0
Positionsgeber	M: TTL links U_1 (graues Kabel)	PH7, PH6 und PT1 Digitaleingänge Port H.6/7 und Timer-Kanal 1
Positionsgeber	M: TTL rechts U_2 (gelbes Kabel)	PH4, PH5 und PT0 Digitaleingänge Port H.4/5 und Timer-Kanal 0
ADC–Eingang	A: Tachogenerator–Ausgang (schwarzes Kabel)	PAD08 ADC-Kanal 0 (an ATD1)
ADC–Eingang	A: Stromsensor–Ausgang (schwarzes Kabel)	PAD09 ADC-Kanal 1 (an ATD1)
Relais	A: Rotes Kabel	PB0 Digitalausgang Port B.0
	Auf Dragon12	LEDs an Port B.7...B.1 Debug-Hilfen
	Auf Dragon12	Buttons an Port H.3...H.0 Anwendereingaben
		RTI-Timer erzeugt Abtastzeit
Verbindung zum PC	Auf Dragon12	Serielle Schnittstelle SCI0 Codewarrior–Debugger

Tabelle 5: Verdrahtung zwischen Mikrocontroller und Motorboard bzw. Anpassung